

UNIVERZA V LJUBLJANI
FAKULTETA ZA MATEMATIKO IN FIZIKO
FAKULTETA ZA RAČUNALNIŠTVO IN INFORMATIKO

Računalništvo in matematika – 2. stopnja

Matic Pokorn

**NEWTONOVA METODA ZA DVONIVOJSKO
OPTIMIZACIJO TRAJEKTORIJ
DIFERENCIALNO PLOŠČATIH SISTEMOV Z
UPORABO ANALITIČNIH ODVODOV**

Magistrsko delo

Mentor: izr. prof. dr. Jan Grošelj

Ljubljana, 2026

Zahvala

Neobvezno. Zahvaljujem se ...

Kazalo

Seznam kratic	1
1 Uvod	1
2 Pregled področja	3
2.1 Diferencialno ploščati sistemi	3
2.2 Optimizacija trajektorij brezpilotnikov	4
2.3 Dvonivojska optimizacija	5
2.4 Newtonova metoda	6
3 Opis metode	6
3.1 Problem s podanimi časi segmentov	6
3.2 Prevedba na QP	7
3.2.1 Izračun matrike $\mathbf{Q}$	7
3.2.2 Izračun matrike $\mathbf{A}$ in vektorja $\mathbf{b}$	9
3.3 Problem s časovnimi spremenljivkami	9
3.4 Odvajanje s KKT pogoji	11
3.4.1 KKT pogoji QP	12
3.4.2 Izrek o implicitni funkciji	13
3.4.3 Računanje gradienta	14
3.4.4 Računanje Hessove matrike	14
3.5 Newtonova metoda in optimizacijski algoritem	15
3.5.1 Gradientni korak	15
3.5.2 Newtonov korak	15
3.5.3 Optimizacijski algoritem	15
3.6 Pohitritve	18
3.6.1 Faktorizacija KKT matrike $\mathbf{K}$	18
3.6.2 Računanje desne strani linearnih sistemov enačb	18
3.6.3 Pasovna struktura Hessove matrike	21
4 Eksperimenti	22
4.1 Implementacija in uporaba zunanjih knjižnic	22
4.2 Konvergenca	23
4.3 Pasovna struktura Hessove matrike	25
4.4 Hitrost metode v realnem času	28
5 Zaključek	30
A Primer	33
A.1 QP	33
A.2 Izračun gradienta	34
A.3 Projekcija gradienta in gradientni korak	37

B	Podrobnosti implementacije	37
B.1	Einsteinova sumacijska notacija	38
B.2	Vektorizacija vhodnih podatkov	38
B.3	Izračun gradienta	40

Newtonova metoda za dvonivojsko optimizacijo trajektorij diferencialno ploščatih sistemov z uporabo analitičnih odvodov

POVZETEK

Za načrtovanje zveznih trajektorij diferencialno ploščatih sistemov, kot so kvadrotorji, je eden izmed ustaljenih načinov dvonivojska optimizacija, kjer je spodnji nivo definiran kot kvadratični program (QP), zgornji nivo pa se rešuje z izračunom gradienta optimalne rešitve spodnjega nivoja; tega se s pomočjo občutljivostne analize KKT pogojev QP in izreka o implicitni funkciji da izračunati analitično. V tem delu naredimo korak dlje in analitično izračunamo parcialne odvode drugega reda optimalne rešitve QP, kar omogoči optimizacijo z Newtonovo metodo in posledično konvergenco k optimalni rešitvi v bistveno manj korakih kot z obstoječimi pristopi. Z izkoriščenjem strukture matrik cenovne funkcije in linearnih omejitev dosežemo dodatne pohitritve.

A Newton method for bilevel trajectory optimization of differentially flat systems with analytical derivatives

ABSTRACT

Bilevel optimization is an established approach to continuous trajectory planning for differentially flat systems such as quadrotors, where the lower level is defined as a quadratic program (QP) and the upper level is solved with computing the gradient of the optimal solution of the lower level. The gradient can be computed analytically via the sensitivity analysis of the QP's KKT conditions and use of the implicit function theorem. In this work we take a step further and analytically compute second order derivatives of the QP optimal solution, enabling the use of the Newton method and consequently convergence to the optimum in significantly less iterations than with existing approaches. Utilising the structures of the cost and constraint matrices we achieve additional speed-ups.

Math. Subj. Class. (2020): 90C53, 90C30, 90C20, 90C55, 49M15, 90C31

Ključne besede: dvonivojska optimizacija, Newtonova metoda, diferencialno ploščati sistemi, polinomski zlepci

Keywords: bilevel optimization, Newton method, differentially flat systems, polynomial splines

Seznam kratic

DFBC krmilniki osnovani na diferencialni ploščatosti (differential flatness based control)

IIF izrek o implicitni funkciji (implicit function theorem)

KKT Karush-Kuhn-Tucker pogoji (Karush-Kuhn-Tucker conditions)

LQR linearni kvadratni regulator (linear quadratic regulator)

MPC modelno prediktivno krmiljenje (model predictive control)

MPEC matematično programirane z ravnovesnimi pogoji (mathematical programming with equilibrium constraints)

PID proporcionalno-integralsko-odvajalni krmilnik (proportional-integral-derivative controller)

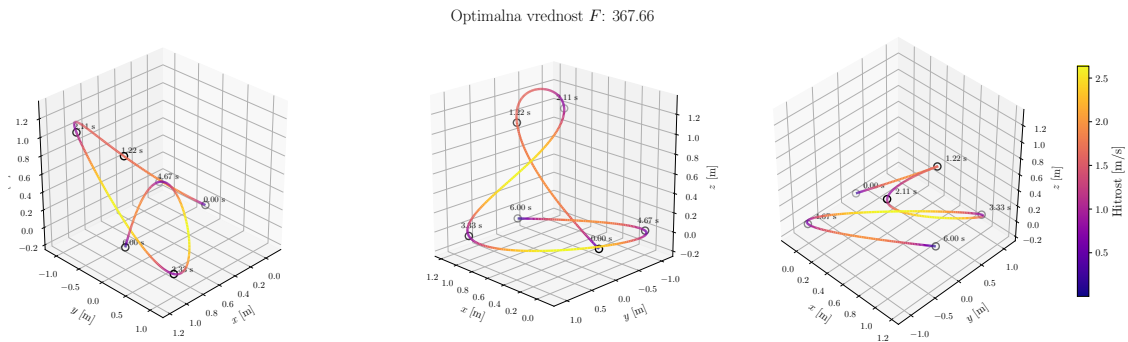
QP kvadratični program (quadratic program)

1 Uvod

V zadnjih letih smo priča hitremu razvoju na področju avtonomnega upravljanja brezpilotnih letalnikov, kjer se prepletajo različna raziskovalna področja, kot so računalniški vid, računska geometrija, teorija upravljanja, numerične metode, optimizacija, itd. Načrtovanje gibanja brezpilotnikov se v grobem deli na tri bolj ali manj ločene procese: (i) *predstavitev okolja*, (ii) *načrtovanje poti* in (iii) *optimizacijo trajektorij* [zhou2025survey]. Predstavitev okolja se ukvarja s prepoznavanjem in predstavitvijo ovir s pomočjo senzorjev, kot so LiDAR in kamere za zaznavanje globine. Načrtovanje poti se ukvarja z iskanjem geometrične poti, ki se izogne oviram, ki so bile določene s predstavitvijo okolja. Optimizacija trajektorij pa iz določene geometrijske poti izračuna za brezpilotnika dopustno trajektorijo, pri čemer optimizira želen kriterij in se izogiba oviram. Pomembno vlogo igra tudi *vodenje sistemov*, ki se ukvarja z raziskovanjem krmilnikov, ki iz zelene trajektorije izračunajo pripadajoče krmilne vhode (npr. hitrosti vrtenja propelerjev). Za nekatere oblike brezpilotnih letalnikov, kot so kvadrotorji, velja, da so diferencialno ploščati sistemi [mellinger2011minimum], kar pomeni, da lahko krmilne vhode sistema opišemo z izhodom sistema in njegovimi odvodi.

Za avtonomno letenje, predvsem v okoljih z veliko ovirami in pri večjih hitrostih, kjer je potrebno izvajanje agresivnih manevrov, je bistvenega pomena generiranje dopustnih trajektorij, pri čemer optimiziramo želen kriterij, hkrati pa se brezpilotnik izogne fizičnim preprekam, kot so npr. stene, stebri, drevesa, itd. Prav tako je pomembno, da so algoritmi, ki take probleme rešujejo, izjemno hitri, posebno v primerih, kjer se okolje brezpilotnika hitro spreminja (npr. imamo premikajoče se ovire) in je treba trajektorije računati v realnem času. V tem magistrskem delu rešujemo problem optimizacije trajektorije brezpilotnika, kjer imamo podan seznam točk v prostoru $P_0, P_1, \dots, P_n$, skozi katere mora brezpilotnik leteti, ter čas letenja T .

Pri tem želimo minimizirati povprečen trzaj (3. odvod) ali kak drug odvod trajektorije. Na Sliki 1 je predstavljen primer optimalne trajektorije z 11 vmesnimi točkami. Medtem ko se da problem, kjer so časi po posameznih segmentih $\mathbf{t} = [t_1, t_2, \dots, t_n]$ fiksni, reformulirati kot QP (quadratic program, kvadratični program), je naš problem nekonvexen. Tega lahko rešimo tako, da najdemo optimalni $\mathbf{t}$ z iterativnim reševanjem problema s fiksnimi časi, kjer se z vsako iteracijo približamo minimumu. Ta pristop je prvič predstavljen v seminalnem članku [mellinger2011minimum], vendar tu problem s spremenljivimi časi segmentov rešijo z numeričnim izračunom gradienta, kar je računsko zahtevno, saj zahteva več evaluacij cenovne funkcije. V [sun2021fast] z metodo računanja analitičnega gradienta z občutljivostno analizo KKT (Karush-Kuhn-Tucker conditions, Karush-Kuhn-Tucker pogoji) oz. dvonivojsko optimizacijo dosežejo pohitritve. V tem delu na podoben način izračunamo tudi analitične parcialne odvode drugega reda, kar nam omogoči uporabo Newtonove metode, ki konvergira k optimalni rešitvi v manj korakih kot v [sun2021fast].



Slika 1: Primer optimalne trajektorije glede na trzaj (3. odvod) in s končnim časom 6 s, kjer so s črnimi krogi podane vmesne točke, ob njih pa so pripisani pripadajoči vmesni časi.

Doprinosi tega magistrskega dela so sledeči:

- razvijemo metodo računanja drugih parcialnih odvodov optimalne rešitve spodnjega nivoja dvonivojskega optimizacijskega problema, kar nam omogoči uporabo Newtonove metode za optimizacijo zgornjega nivoja;
- izkoristimo bločno strukturo matrik cenovne funkcije in linearnih omejitev ter njihovih odvodov, da dosežemo dodatne pohitritve;
- z eksperimenti pokažemo, da metoda skonvergira k lokalno optimalni rešitvi v manj korakih kot gradientni spust ne glede na stopnjo polinomov, optimizirane vrednosti ali števila segmentov;
- nadalje ugotovimo, da je Hessova matrika skoraj pasovna in da pri izračunu zgolj nekaj pasov Hessove matrike dosežemo pohitritev metode, pri čemer je vpliv izgube izvenpasovnih elementov Hessove matrike na konvergenco zanemarljiv.

Dostopnost implementacije. Implementacija metode, opisane v tem delu, je javno dostopna na GitHub repozitoriju:

https://github.com/maticpokorn/trajectory_optimization.

Preostanek tega magistrskega dela je strukturiran na sledeči način. Najprej v poglavju 2 opišemo sorodna dela ter dela in pojme, ki služijo kot temelj naši metodi. Nato v poglavju 3 predstavimo optimizacijski problem in podrobno opišemo našo metodo reševanja in jo teoretično podkrepimo. V poglavju 4 analiziramo implementacijo naše metode v smislu konvergence in hitrosti v odvisnosti od vhodnih parametrov. V poglavju 5 povzamemo vsebino in predlagamo izboljšave in nadaljnje delo. V dodatki A predstavimo del naše metode na konkretnem primeru, v dodatku B pa podrobno predstavimo učinkovit izračun parcialnih odvodov cenovne funkcije.

2 Pregled področja

V tem poglavju opišemo sorodna dela in predstavimo pojme in dognanja, na katerih je naše delo osnovano. Najprej podamo opis diferencialno ploščatih sistemov in njihovo povezavo z optimizacijo trajektorij. Nato predstavimo trenutno stanje področja optimizacije trajektorij za kvadrotorje in raziskave, na katerih naše delo bodisi temelji, bodisi predstavlja alternativne pristope k reševanju podobnih problemov. Potem opišemo področje dvonivojske optimizacije, na koncu pa še Newtonovo metodo za iskanje ekstremov.

2.1 Diferencialno ploščati sistemi

V tem delu se opiramo na pojem diferencialne ploščatosti. Sistem upravljanja

$$\dot{x} = f(x, u), \quad x \in \mathbb{R}^n, u \in \mathbb{R}^m \quad (2.1)$$

je diferencialno ploščat, če lahko najdemo tak nabor izhodnih spremenljivk $y \in \mathbb{R}^m$ (ploščati izhodi), da lahko stanje x in krmilni vhod u izrazimo kot funkciji ploščatih izhodov in njihovih odvodov [murray1995differential]:

$$x = \phi(y, \dot{y}, \ddot{y}, \dots, y^{(p)}) \quad (2.2)$$

$$u = \psi(y, \dot{y}, \ddot{y}, \dots, y^{(q)}). \quad (2.3)$$

Diferencialno ploščati sistemi upravljanja so koristni v primerih, kot je naš, kjer želimo eksplicitno določiti trajektorije. Tako lahko iz dovolj gladke trajektorije (izhoda) preprosto zračunamo vhod in stanje sistema. Izkaže se, da so tudi kvadrotorji diferencialno ploščati sistemi; postopek prevedbe iz izhodov na vhode sistema kvadrotorja je opisan v [mellinger2011minimum]. V tem delu se osredotočimo na generiranje optimalnih trajektorij, diferencialna ploščatost pa je pomembna, ker želimo, da sistem trajektoriji dejansko sledi.

Pomemben vidik je tudi konstrukcija krmilnih sistemov za kvadrotorje. Obstajajo različni pristopi, kot so PID (proportional-integral-derivative controller, proporcionalno-integralsko-odvajalni krmilnik) krmilniki, linearni kvadratni regulatorji (LQR), modelno prediktivno krmiljenje (MPC) in krmilniki, osnovani na diferencialni ploščatosti (DFBC) [lee2010geometric]. Za diferencialno ploščate sisteme, kot so kvadrotorji, in agresivne manevre, so najbolj primerni nelinearni MPC krmilniki in

DFBC krmilniki [sun2022comparative], saj lahko upoštevajo omejitve sistema in zagotavljajo stabilnost. MPC krmilniki optimizirajo krmilne vhode na vsakem časovnem koraku glede na napovedano prihodnost, medtem ko DFBC krmilniki neposredno izračunajo krmilne vhode iz trajektorije, kar omogoča hitro odzivanje in sledenje trajektoriji. V zadnjih letih se pojavljajo tudi krmilniki, ki uporabljajo metode strojnega učenja, kot so spodbujevano učenje (reinforcement learning) [hwangbo2017control] ali odvedljivi MPC krmilniki [amos2018differentiable].

V tem delu se osredotočamo na generiranje optimalnih trajektorij, ki so nato lahko uporabljene z različnimi krmilnimi sistemi, vključno z MPC in DFBC krmilniki, odvisno od specifičnih zahtev aplikacije in želene ravni nadzora nad sistemom.

2.2 Optimizacija trajektorij brezpilotnikov

Načrtovanje trajektorij brezpilotnikov s polinomskimi zlepkami se je uveljavilo v [mellinger2011minimum], kjer so problem s fiksno razporeditvijo časa po segmentih definirali kot QP. Kasneje so v [richter2016polynomial] izboljšali numerično stabilnost te metode tako, da so enakostne omejitve vključili v cenovno funkcijo, v [burke2020generating] pa so se osredotočili na pohitritev in pokazali linearno časovno zahtevnost v odvisnosti od števila segmentov. V [mellinger2011minimum] so predstavili tudi problem, kjer optimizirajo časovno razporeditev po segmentih; optimizacijo izvedejo z gradientnim spustom, kjer pa gradient izračunajo z metodo končnih diferenc, ki pa za izračun gradienta zahteva linearno mnogo rešitev QP v razmerju s številom segmentov. Ta problem rešijo v [sun2021fast], kjer gradient izračunajo analitično z občutljivostno analizo KKT QP. V [gao2018online] čase segmentov določijo heuristično. V [wang2021generating] problem s fiksnimi časi segmentov rešijo z dvojnimi opisovanjem polinomov in dosežejo izjemne pohitritve.

V optimizaciji trajektorij pomembno vlogo igrajo neenakostne omejitve za izogibanje oviram in omejevanje maksimalne hitrosti, pospeška, itd. Za izogibanje oviram se pogosto uporabljajo pristopi, ki uporabljajo zaporedja konveksnih poliedrov, da ustvarijo varne koridorje letenja; obstajajo algoritmi, ki učinkovito izračunajo vrčtan polieder okoli točke [deits2015computing] oz. daljice [liu2017planning]. Na ta način neenakostne omejitve za izogibanje oviram uporabijo v [sun2021fast], kjer kasneje optimizirajo časovno razporeditev po segmentih. V [tordesillas2021faster] namesto optimizacije časov segmentov problem formulirajo kot mešan celoštevilski kvadratni program (MIQP), kjer fiksirajo čase segmentov in uporabijo diskretne spremenljivke, da določijo, kateri segmenti se bodo nahajali v katerem poliedru.

Algoritmi, ki optimizacijo polinomskih trajektorij rešujejo s QP, pogosto uporabljajo različne baze polinomov, kot so Bernsteinovi polinomi [sun2021fast, tordesillas2021faster] in B-zlepki [zhou2020ego], kar omogoča enostavno vključevanje neenakostnih omejitev s kontrolnimi točkami in boljšo numerično stabilnost v primerjavi z običajnimi monomskimi polinomi [runge1901]. Kljub prednostim Bernsteinovih polinomov in B-zlepkov tem delu uporabljamo monomske polinome, saj nam to omogoči enostavnejšo izpeljavo analitičnih odvodov drugega reda, kar je ključno za našo metodo.

Obstajajo tudi pristopi, ki ne ločijo časovnih in prostorskih spremenljivk [fridovich2018planning] in se zanašajo na kompleksne algoritme za nelinearno programiranje [wachter2006implementation]. Ti algoritmi pogosto omogočajo sočasno optimizacijo vmesnih točk znotraj določenega dopustnega območja, kar poveča prilagodljivost trajektorije.

Pristopi za optimizacijo trajektorij se srečujejo z dilemo, kjer morajo najti ravnovesje med hitrostjo izračuna in optimalnostjo rezultata. V [zhou2020ego] se osredotočajo na hitrost in trajektorije izračunajo v milisekundah, vendar ne zagotavljajo optimalnosti, kar lahko povzroči bolj agresivne manevre in posledično neučinkovite in nevarne trajektorije. V [richter2016polynomial] se osredotočajo na optimalnost, vendar je čas izračuna trajektorije v sekundah, kar je neprimerno za načrtovanje trajektorij v realnem času. V tem delu zagotavljamo konvergenco k lokalno optimalni rešitvi, pri čemer želimo zagotoviti hitro izračunavanje trajektorij. Medtem ko naša metoda z obstoječo implementacijo ni dovolj hitra za načrtovanje trajektorij povsem v realnem času, je primerna za asinhrono generiranje trajektorij, kjer se trajektorije izračunajo v ločenem procesu, ki ni vezan na časovno omejitev, in se nato posredujejo v proces vodenja brezpilotnika, ki sledi trajektoriji.

2.3 Dvonivojska optimizacija

Dvonivojska optimizacija se ukvarja z optimizacijskimi problemi, kjer je cenovna funkcija enega optimizacijskega problema (zgornji nivo) odvisna od optimalne vrednosti odločitvenih spremenljivk drugega optimizacijskega problema (spodnji nivo). Splošna oblika dvonivojskega optimizacijskega problema je predstavljena kot:

$$\begin{aligned} \min_{x \in X, y^* \in Y} \quad & F(x, y^*) \\ \text{p.p.} \quad & y^* = \arg \min_{y \in Y} \{f(y) : g(x, y) \leq 0\} \\ & G(x, y^*) \leq 0, \end{aligned} \tag{2.4}$$

kjer sta f in g cenovna funkcija in omejitve spodnjega nivoja, F in G pa cenovna funkcija in omejitve zgornjega nivoja.

Znan primer dvonivojske optimizacije je Stacklebergova igra [stackelberg1934marktform], ki je igra med dvema igralcema (voditelj in sledilec), kjer voditelj optimizira svojo strategijo glede na sledilčevo.

Dvonivojska optimizacija v načrtovanju trajektorij brezpilotnikov in v robotiki na splošno se navadno uporablja, ko je optimizacijski problem odvisen od časovne razporeditve in nekaterih drugih parametrov, ki pa so tudi odvisni od časovne razporeditve. V našem primeru so to koeficienti polinomov. Pogosto se, kot v našem primeru, izkaže, da se da spodnji nivo izraziti kot QP, ki se ga da rešiti s standardnimi knjižnicami. Tudi problem v [mellinger2011minimum] je implicitno definiran kot dvonivojski optimizacijski problem. V [sun2021fast] poleg izračuna analitičnih gradientov svojo rešitev podkrepijo teoretično. Izven domene načrtovanja trajektorij v [amos2017optnet] dvonivojsko optimizacijo na QP uporabijo za integracijo QP kot plast v golobkih nevronske mreže, kjer so parametri QP odvisni od drugih plasti v nevronske mreži. V [dyro2022second] razvijejo teorijo računanja Hessove matrike zgornjega problema splošnih dvonivojskih neomejenih optimizacijskih problemov z uporabo izrek o implicitni funkciji (IIF); z eksperimenti tudi pokažejo hitrejšo konvergenco kot obstoječe metode.

Poleg robotike se dvonivojska optimizacija aplikativno uporablja tudi v ekonomiji

2.4 Newtonova metoda

Newtonova metoda je iterativna metoda za iskanje ekstremov dvakrat zvezno odvedljivih funkcij [newton1736method, boyd2004convex]. Korak v iteraciji Newtonove metode za funkcije več spremenljivk se izračuna z enačbo

$$x_{k+1} = x_k - [\nabla_f^2(x_k)]^{-1} \nabla_f(x_k), \quad (2.5)$$

kjer je $f : \mathbb{R}^n \rightarrow \mathbb{R}$ dvakrat zvezno odvedljiva funkcija, $x_k \in \mathbb{R}^n$ trenutna točka, $x_{k+1} \in \mathbb{R}^n$ pa naslednja točka iteracije. $\nabla_f^2(x_k)$ predstavlja Hessovo matriko funkcije f v točki x_k , $\nabla_f(x_k)$ pa gradient funkcije f v točki x_k . Kot že omenjeno, v [dyro2022second] razvijejo teorijo računanja gradienta in Hessove matrike za neomejene dvonivojske optimizacijske probleme in pokažejo hitro konvergenco. Tudi v tem delu eksperimenti pokažejo zelo hitro konvergenco k optimalni rešitvi.

3 Opis metode

V tem poglavju opišemo našo metodo. Najprej definiramo osnovni problem v obliki minimizacije funkcionala z omejitvami s fiksnimi potovalnimi časi po segmentih. Nato pokažemo, kako se problem minimizacije funkcionala prevede na QP, kjer optimizacijske spremenljivke postanejo koeficienti polinomov. Dodamo optimizacijo po časih segmentov in predstavimo glavni optimizacijski problem, ki ga v tem delu rešujemo. Nato opišemo, kako lahko s pomočjo KKT pogojev QP izluščimo gradient in Hessovo matriko cenovne funkcije v zgornjem nivoju dvonivojskega problema. Zatim predstavimo minimizacijski algoritem, ki sloni na Newtonovi metodi. Na koncu pokažemo, kako lahko z uporabo bločne strukture matrik in strukture Hessove matrike dosežemo velike pohitritve.

Najprej pa uvedemo nekaj nestandardne notacije, ki bo služila za boljšo preglednost in razumljivost v enačbah v tem poglavju:

Naj bo $\mathbf{A}(\mathbf{t}) \in \mathcal{C}^2(\mathbb{R}^k, \mathbb{R}^{m \times n})$ matrika, kjer je vsak element dvakrat zvezno odvedljiva funkcija vektorja $\mathbf{t}$.

- S simbolom $\partial_{t_i} \mathbf{A}$ označimo matriko parcialnih odvodov glede na i -to komponento vektorja $\mathbf{t}$:

$$\partial_{t_i} \mathbf{A} := \frac{\partial \mathbf{A}(\mathbf{t})}{\partial t_i} = \left[\frac{\partial a_{uv}(\mathbf{t})}{\partial t_i} \right]_{u,v=1}^{m,n} \quad (3.1)$$

- V nadaljevanju bomo zaradi večje preglednosti izpeljav pogosto opuščali eksplisitno navajanje argumenta $\mathbf{t}$ (npr. $\mathbf{A}$ namesto $\mathbf{A}(\mathbf{t})$), razen v primerih, ko je to nujno za razumevanje.

3.1 Problem s podanimi časi segmentov

Rešujemo spodnji problem:

Definirano imamo zaporedje vmesnih točk $P_0, \dots, P_n$ in potovalne čase za posamezen segment $t_1, \dots, t_n$. Najti moramo polinome $p_1, \dots, p_n$, ki minimizirajo spodnji funkcional:

$$\begin{aligned}
& \min_{p_1, \dots, p_n} \sum_{i=1}^n \int_0^{t_i} (p_i^{(r)}(\tau))^2 d\tau \\
& \text{p.p. } p_1(0) = P_0, \quad p_n(t_n) = P_n \\
& \quad p_1^{(k)}(0) = 0 \quad ; \quad k = 1, \dots, K \\
& \quad p_n^{(k)}(t_n) = 0 \quad ; \quad k = 1, \dots, K \\
& \quad p_i(t_i) = p_{i+1}(0) = P_i \quad ; \quad i = 1, \dots, n-1 \\
& \quad p_i^{(k)}(t_i) = p_{i+1}^{(k)}(0) \quad ; \quad i = 1, \dots, n-1, \quad k = 1, \dots, K
\end{aligned} \tag{3.2}$$

Pravzaprav tak problem rešujemo trikrat, posebej za vsako od osi x , y ter z , kjer t_i ostane isti, spremenijo pa se zgolj koordinate vmesnih točk P_i za posamezno os. Z nekaj svobode pa lahko izberemo parametre:

- d , stopnja polinomskih zlepkov
- k , red zveznosti trajektorije
- r , optimizacijska stopnja odvoda

Veljati mora $r \leq k \leq d$, standardne vrednosti pa so $d = 9$, $r = 3$ (trzaj) ali $r = 4$ (pok) ter $k = r$.

3.2 Prevedba na QP

V tem razdelku opišemo, kako problem, predstavljen v (3.2) prevedemo na QP oblike:

$$\begin{aligned}
& \frac{1}{2} \mathbf{a}^\top \mathbf{Q} \mathbf{a} \\
& \text{p.p. } \mathbf{A} \mathbf{a} = \mathbf{b},
\end{aligned} \tag{3.3}$$

kjer je $\mathbf{a} \in \mathbb{R}^{n(d+1)}$ optimizacijski vektor in predstavlja koeficiente polinomskih zlepkov, $\mathbf{Q} \in \mathbb{R}^{n(d+1) \times n(d+1)}$ je simetrična in pozitivno definitna matrika, $\mathbf{A} \in \mathbb{R}^{M \times n(d+1)}$ in $\mathbf{b} \in \mathbb{R}^M$ pa predstavljata linearne omejitve. Najprej bomo pokazali, kako izračunamo matriko $\mathbf{Q}$, nato pa še, kako izračunamo matriko $\mathbf{A}$ in vektor $\mathbf{b}$. Bistvenega pomena pri tej izpeljavi je, da so elementi matrik $\mathbf{Q}$ in $\mathbf{A}$ zvezno odvisni od vmesnih časov t_i .

3.2.1 Izračun matrike $\mathbf{Q}$

Matriko $\mathbf{Q}$ izračunamo tako, da ločeno prevedemo posamezen integral $\int_0^{t_i} (p_i^{(r)}(\tau))^2 d\tau$ na matrično enačbo $\mathbf{a}_i^\top \mathbf{Q}_i \mathbf{a}_i$, kjer je $\mathbf{a}_i$ vektor koeficientov i -tega polinoma. Celotno vsoto $\sum_{i=1}^n \int_0^{t_i} (p_i^{(r)}(\tau))^2 d\tau$ pa nato predstavimo kot bločno matriko

$$\mathbf{a}^\top \mathbf{Q} \mathbf{a} = \begin{bmatrix} \mathbf{a}_1 \\ \mathbf{a}_2 \\ \vdots \\ \mathbf{a}_n \end{bmatrix}^\top \begin{bmatrix} \mathbf{Q}_1 & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & \mathbf{Q}_2 & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & \mathbf{Q}_n \end{bmatrix} \begin{bmatrix} \mathbf{a}_1 \\ \mathbf{a}_2 \\ \vdots \\ \mathbf{a}_n \end{bmatrix} \tag{3.4}$$

Za preoblikovanje integrala $J := \int_0^{t_i} (p_i^{(r)}(\tau))^2 d\tau$ v matrično obliko uporabimo naslednja rezultata:

1. r -ti odvod polinoma.

$$p^{(r)}(\tau) = \sum_{k=r}^d \left[\prod_{m=0}^{r-1} (k-m) \right] a_k \tau^{k-r} \quad (3.5)$$

2. Kvadrat polinoma.

$$p(\tau)^2 = \sum_{k=0}^{2d} \left(\sum_{l=0}^d a_l a_{k-l} \right) \tau^k \quad (3.6)$$

Tu predpostavimo, da so koeficienti, kjer je indeks večji od stopnje polinoma ali manjši od 0, enaki 0.

Z združitvijo zgornjih rezultatov lahko izrazimo integrand $(p^{(r)}(\tau))^2$ kot:

$$(p^{(r)}(\tau))^2 = \sum_{k=0}^{2d} \left(\sum_{l=r}^d \prod_{m=0}^{r-1} (l-m)(k-l-m) a_l a_{k-l} \right) \tau^{k-2r} \quad (3.7)$$

Sedaj pa lahko izračunamo integral:

$$\int_0^{t_i} (p^{(r)}(\tau))^2 d\tau = \sum_{k=0}^{2d} \left(\sum_{l=r}^d \prod_{m=0}^{r-1} (l-m)(k-l-m) a_l a_{k-l} \right) \frac{t_i^{k-2r+1}}{k-2r+1} \quad (3.8)$$

Sedaj že lahko opazimo, da imajo posamezni členi v vsoti obliko $\mathbf{Q}_{uv} a_u a_v$ za neke $\mathbf{Q}_{uv}$. Vendar pa koeficienti a_u in a_v nastopajo v različnih členih vsote. Zato $\mathbf{Q}_{uv}$ izračunamo tako, da izračunamo parcialne odvode drugega reda funkcije J . S tem členi vsote, ki ne vsebujejo $a_u a_v$, postanejo ničelni, medtem ko vsota členov, ki vsebujejo $a_u a_v$, postane $\mathbf{Q}_{uv}$:

$$\mathbf{Q}_{uv} = \frac{\partial^2 J}{\partial a_u \partial a_v} \quad (3.9)$$

Najprej izračunamo parcialni odvod prvega reda funkcije J :

$$\begin{aligned} \frac{\partial J}{\partial a_u} &= \sum_{k=0}^{2d} \sum_{l=0}^d \prod_{m=0}^{r-1} (l-m)(k-l-m) \left(\frac{\partial a_l}{\partial a_u} a_{k-l} + \frac{\partial a_{k-l}}{\partial a_u} a_l \right) \frac{t^{k-2r+1}}{k-2r+1} \\ &= 2 \sum_{k=0}^{2d} \prod_{m=0}^{r-1} (u-m)(k-u-m) a_{k-u} \frac{t^{k-2r+1}}{k-2r+1} \end{aligned} \quad (3.10)$$

Pri odvajanju notranje vsote dobimo samo dva neničelna člena, kjer je $0 \leq u \leq d$. Sedaj dobljeno funkcijo odvajamo še enkrat po a_v :

$$\begin{aligned} \frac{\partial^2 J}{\partial a_u \partial a_v} &= 2 \sum_{k=0}^{2d} \prod_{m=0}^{r-1} (v-m)(k-v-m) \frac{\partial a_{k-v}}{\partial a_u} \frac{t^{k-2r+1}}{k-2r+1} \\ &= 2 \prod_{m=0}^{r-1} (v-m)(u-m) \frac{t^{v+u-2r+1}}{v+u-2r+1} \end{aligned} \quad (3.11)$$

Pri drugem parcialnem odvodu preostale vsote pa pa dobimo samo en neničelen člen, kjer $v = k - u$ in $0 \leq v \leq d$.

Torej lahko zaključimo, da velja:

$$\mathbf{Q}_{uv} = \begin{cases} \prod_{m=0}^{r-1} (u-m)(v-m) \frac{t^{u+v-2r+1}}{u+v-2r+1} & u, v \geq r \\ 0 & \text{sicer} \end{cases}, \quad \text{in} \quad \mathbf{Q}_{vu} = \mathbf{Q}_{uv} \quad (3.12)$$

,

funkcijo J pa lahko izrazimo kot $J = \sum_{uv} \mathbf{Q}_{uv} a_u a_v = \mathbf{a}^\top \mathbf{Q} \mathbf{a}$.

S tem smo preoblikovali ciljni funkcional v kvadratno funkcijo koeficientov polinoma.

3.2.2 Izračun matrike $\mathbf{A}$ in vektorja $\mathbf{b}$

Matriko $\mathbf{A}$ in vektor $\mathbf{b}$ izračunamo tako, da prevedemo linearne omejitve problema (3.2) na matrično obliko $\mathbf{A}\mathbf{a} = \mathbf{b}$. Vsaka linearna omejitev v problemu (3.2) se prevede v vrstico matrike $\mathbf{A}$ in ustrezen element vektorja $\mathbf{b}$. Tako kot matrika $\mathbf{Q}$ ima tudi $\mathbf{A}$ bločno strukturo, in sicer:

$$\mathbf{A}(\mathbf{t}) = \begin{bmatrix} \mathbf{A}_1(0) & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{0} \\ \mathbf{A}_1(t_1) & -\mathbf{A}_2(0) & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{A}_2(t_2) & -\mathbf{A}_3(0) & \cdots & \mathbf{0} & \mathbf{0} \\ \vdots & \vdots & \ddots & \ddots & \vdots & \vdots \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{A}_{n-1}(t_{n-1}) & \mathbf{A}_n(0) \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{A}_n(t_n) \end{bmatrix} \quad (3.13)$$

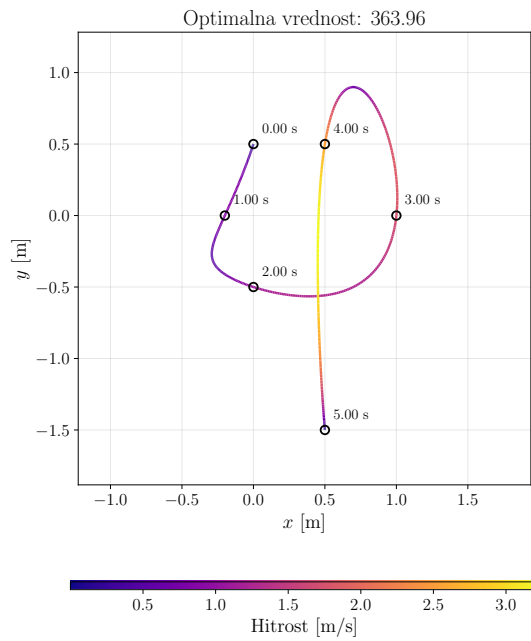
Sedaj pokažemo, kako so strukturirane posamezne komponente matrike $\mathbf{A}$. Ločeno pozornost posvetimo začetnemu bloku $\mathbf{A}_1(0)$, končnemu bloku $\mathbf{A}_n(t_n)$ in vmesnim parom blokov $\mathbf{A}_i(t_i)$ in $-\mathbf{A}_{i+1}(0)$, kjer $i = 1, \dots, n-1$:

- Začetni blok $\mathbf{A}_1(0)$ predstavlja omejitve začetka trajektorije, ki zahtevajo, da $p_1(0) = P_0$ in $p_1^{(k)}(0) = 0$ za $k = 1, \dots, K$.
- Končni blok $\mathbf{A}_n(t_n)$ predstavlja omejitve konca trajektorije, ki zahtevajo, da $p_n(t_n) = P_n$ in $p_n^{(k)}(t_n) = 0$ za $k = 1, \dots, K$.
- Vmesni pari blokov $\mathbf{A}_i(t_i)$ in $-\mathbf{A}_{i+1}(0)$ predstavljajo omejitve zveznosti trajektorije na vmesnih točkah, ki zahtevajo, da $p_i(t_i) = p_{i+1}(0) = P_i$ in $p_i^{(k)}(t_i) = p_{i+1}^{(k)}(0)$ za $k = 1, \dots, K$ in $i = 1, \dots, n-1$.

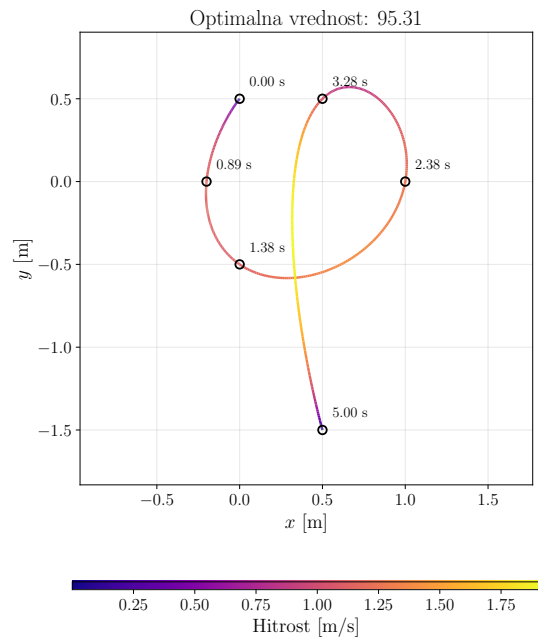
Podrobnejši primer konstrukcije QP s fiksnimi časi segmentov je podan v dodatku A.

3.3 Problem s časovnimi spremenljivkami

V prejšnjem poglavju smo predstavili problem, kjer so časi segmentov podani kot fiksni parametri problema. Pogosto pa nam je pomemben le celoten čas potovanja po trajektoriji. Na primer, slika 2 nudi primerjavo med fiksno določenimi vmesnimi časi



(a) Trajektorija s fiksno določenimi vmesnimi časi (vsak segment 1s) - izračunana s QP.



(b) Trajektorija z optimiziranimi vmesnimi časi - izračunana z dvonivojsko optimizacijo.

Slika 2: Primerjava trajektorij, kjer fiksno določimo vmesne čase (a), in kjer izvedemo optimizacijo po vmesnih časih (b). Obe trajektoriji imata končni čas 5s, vendar ima desna trajektorija nižjo optimalno vrednost in tudi nižjo maksimalno hitrost.

in optimiziranimi vmesnimi časi za isto zaporedje vmesnih točk in isti skupen čas. Če nam je vseeno, kakšni so časi na vmesnih točkah, je očitno, da optimizacija časov segmentov vodi do bistveno boljše trajektorije. Zato lahko definiramo želen skupen čas trajektorije T in vektor spremenljivk $\mathbf{t} = [t_1, t_2, \dots, t_n]$, kjer velja $\sum_{i=1}^n t_i = T$ in $t_i > 0$. Sedaj lahko formuliramo nov optimizacijski problem:

$$\begin{aligned} \min_{\mathbf{t}, \mathbf{a}} \quad & \frac{1}{2} \mathbf{a}^\top \mathbf{Q}(\mathbf{t}) \mathbf{a} \\ \text{p.p.} \quad & \mathbf{A}(\mathbf{t}) \mathbf{a} = \mathbf{b} \\ & \mathbf{t} > \mathbf{0} \\ & \mathbf{1}^\top \mathbf{t} = T, \end{aligned} \tag{3.14}$$

kjer je $\mathbf{1}$ vektor enic. Očitno je, da ta problem ni konveksen, niti nima konveksnih omejitev. Zato ga je koristno reformulirati v sledečo ekvivalentno dvonivojsko obliko:

$$\begin{aligned} \min_{\mathbf{t}, \mathbf{a}^*} \quad & F(\mathbf{t}) := \frac{1}{2} \mathbf{a}^{*\top} \mathbf{Q}(\mathbf{t}) \mathbf{a}^* \\ \text{p.p.} \quad & \mathbf{a}^* = \arg \min_{\mathbf{a}} \left\{ \frac{1}{2} \mathbf{a}^\top \mathbf{Q}(\mathbf{t}) \mathbf{a} : \mathbf{A}(\mathbf{t}) \mathbf{a} = \mathbf{b} \right\} \\ & \mathbf{t} > \mathbf{0} \\ & \mathbf{1}^\top \mathbf{t} = T, \end{aligned} \tag{3.15}$$

kjer problemu računanja $\mathbf{a}^*$ rečemo spodnji nivo dvonivojskega optimizacijskega problema, problemu minimizacije $F(\mathbf{t})$ pa zgornji nivo. Če fiksiramo $\mathbf{t}$, je spodnji nivo preprost QP z enakostnimi omejitvami. S takim problemom se srečujejo v [mellinger2011minimum, sun2021fast], kjer v neki točki $\mathbf{t}'$ izračunajo gradient $\nabla_{\mathbf{t}} F(\mathbf{t}')$ in izvedejo gradientni spust. Pri tem za izračun gradienta v [mellinger2011minimum] uporabijo metodo končnih diferenc, v [sun2021fast] pa ga izračunajo analitično.

To je glavni optimizacijski problem, s katerim se v tem magistrskem delu ukvarjamo in preostanek tega poglavja je posvečen opisu naše metode njegovega reševanja. V naslednjem podpoglavju s pomočjo KKT pogojev in izreka o implicitni funkciji pokažemo, da za vsak $\mathbf{t}$ obstaja funkcija $\bar{\mathbf{a}} \in \mathcal{C}^\infty$, kjer $\bar{\mathbf{a}}(\mathbf{t}) = \mathbf{a}^*$. S tem pokažemo, da velja tudi $F \in \mathcal{C}^\infty$ in omogočimo izračun analitičnih parcialnih odvodov višjega reda za $F(\mathbf{t})$ in s tem gradienta in Hessove matrike.

3.4 Odvajanje s KKT pogoji

V tem podpoglavju bomo predstavili, kako s pomočjo KKT izračunamo gradient in Hessovo matriko cenovne funkcije zgornjenivojskega problema. Najprej predstavimo izrek o KKT za splošen optimizacijski program in opišemo lastnosti KKT, če imamo konveksen optimizacijski program. Nato v 3.23 definiramo KKT matriko za QP z enakostnimi omejitvami. V 3.4.2 navedemo izrek o implicitni funkciji in pokažemo, da obstaja analitična funkcija $\bar{\mathbf{a}}$, kjer velja $\bar{\mathbf{a}}(\mathbf{t}) = \mathbf{a}^*$. S tem argumentiramo izračun parcialnih odvodov $\bar{\mathbf{a}}$ ter gradienta in Hessove matrike, ki jih predstavimo v 3.4.3 in 3.4.4.

Spodaj navedemo standarden izrek o KKT [bertsekas1997nonlinear].

Karush-Kuhn-Tuckerjevi pogoji. Za optimizacijski program

$$\min_x f(x) \quad \text{p.p.} \quad g_i(x) \leq 0, \quad i = 1, \dots, m, \quad h_j(x) = 0, \quad j = 1, \dots, p,$$

so Karush-Kuhn-Tuckerjevi pogoji naslednji: obstajajo Lagrangeevi multiplikatorji $\lambda \in \mathbb{R}_{\geq 0}^m$, $\mu \in \mathbb{R}^p$, da velja

$$\nabla f(x) + \sum_{i=1}^m \lambda_i \nabla g_i(x) + \sum_{j=1}^p \mu_j \nabla h_j(x) = 0, \quad (3.16)$$

$$\lambda_i g_i(x) = 0, \quad i = 1, \dots, m, \quad (3.17)$$

$$g_i(x) \leq 0, \quad i = 1, \dots, m, \quad (3.18)$$

$$h_j(x) = 0, \quad j = 1, \dots, p, \quad (3.19)$$

$$\lambda \geq 0. \quad (3.20)$$

Za *konveksne programe*, kjer je f konveksna, g_i konveksne, h_j affine in velja Slaterjev pogoj, postanejo KKT *potrebni in zadostni* za globalno optimalnost. Slaterjev pogoj predstavimo spodaj.

Slaterjev pogoj [bertsekas1997nonlinear, boyd2004convex]. Za konveksni program

$$\min_x f(x) \quad \text{p.p.} \quad g_i(x) \leq 0, \quad i = 1, \dots, m, \quad h_j(x) = 0, \quad j = 1, \dots, p,$$

Slaterjev pogoj velja, če obstaja x° tako, da

- $g_i(x^\circ) < 0$ za vse $i = 1, \dots, m$ (stroga neenakostna dopustnost),
- $h_j(x^\circ) = 0$ for all $j = 1, \dots, p$ (enakostna dopustnost).

Opazimo lahko, da v primeru, kjer neenakostnih omejitev ni, Slaterjev pogoj velja.

3.4.1 KKT pogoji QP

V primeru našega optimizacijskega problema, kjer imamo kvadratično cenovno funkcijo in enakostne omejitve, lahko privzamemo Slaterjev pogoj, saj nimamo neenakostnih omejitev, KKT pa se zreducirajo na:

$$\begin{aligned} \nabla \left(\frac{1}{2} \mathbf{a}^{*\top} \mathbf{Q} \mathbf{a}^* \right) + \sum_{i=1}^m \mu_i^* \nabla (\mathbf{A}_i \mathbf{a}^*) &= \mathbf{0} \\ \mathbf{A}_i \mathbf{a}^* - \mathbf{b}_i &= 0, \quad i = 1, \dots, m, \end{aligned} \quad (3.21)$$

še bolj kompaktno pa jih lahko zapišem kot:

$$\begin{aligned} \mathbf{Q} \mathbf{a}^* + \mathbf{A}^\top \boldsymbol{\mu}^* &= \mathbf{0} \\ \mathbf{A} \mathbf{a}^* &= \mathbf{b}. \end{aligned} \quad (3.22)$$

Pri optimalnosti torej velja naslednja matrična enačba, kjer so μ enakostni Lagrangeevi multiplikatorji, $\mathbf{a}^*$, ter $\boldsymbol{\mu}^*$ pa so vrednosti $\mathbf{a}$ ter μ pri optimalnosti:

$$\underbrace{\begin{bmatrix} \mathbf{Q} & \mathbf{A}^\top \\ \mathbf{A} & \mathbf{0} \end{bmatrix}}_{\mathbf{K}} \underbrace{\begin{bmatrix} \mathbf{a}^* \\ \boldsymbol{\mu}^* \end{bmatrix}}_{\mathbf{y}^*} - \underbrace{\begin{bmatrix} \mathbf{0} \\ \mathbf{b} \end{bmatrix}}_{\mathbf{c}} = \mathbf{0} \quad (3.23)$$

Spodnji optimizacijski problem v enačbi (3.15) bi torej lahko nadomestili z zgornjo matrično enačbo. V literaturi se področju, ki se ukvarja s takimi problemi, reče tudi matematično programirane z ravnovesnimi pogoji.

V nadaljevanju bomo izmenično uporabljali tako osnovno kot kompaktno obliko ($\mathbf{K}\mathbf{y}^* - \mathbf{c} = \mathbf{0}$) zapisa KKT. Kompaktno obliko bomo uporabili, kjer bo to pripomoglo k boljši preglednosti, osnovno obliko pa tam, kjer bo pomembna struktura posameznih delov matrike $\mathbf{K}$, ali pa bo treba ločiti med primarnimi in dualnimi komponentami $\mathbf{y}^*$.

3.4.2 Izrek o implicitni funkciji

Z uporabo izreka o implicitni funkciji lahko pokažemo, da obstaja odprta množica U in gladke funkcije $\gamma : U \rightarrow \mathbb{R}^{m+n}$, $\bar{\mathbf{a}} : U \rightarrow \mathbb{R}^n$, $\bar{\mu} : U \rightarrow \mathbb{R}^m$, kjer velja $\gamma(\mathbf{t}) = (\bar{\mathbf{a}}(\mathbf{t}), \bar{\mu}(\mathbf{t})) = (\mathbf{a}^*, \boldsymbol{\mu}^*)$. To nam da formalno podlago, da cenovno funkcijo $F(\mathbf{t})$ odvajamo glede na $\mathbf{t}$, saj jo lahko zapišemo kot:

$$F(\mathbf{t}) = \frac{1}{2} \mathbf{a}^{*\top} \mathbf{Q}(\mathbf{t}) \mathbf{a}^* = \frac{1}{2} \bar{\mathbf{a}}(\mathbf{t})^\top \mathbf{Q}(\mathbf{t}) \bar{\mathbf{a}}(\mathbf{t}) \quad (3.24)$$

Dodatno velja:

$$\mathbf{K}(\mathbf{t})(\mathbf{y})^* - \mathbf{c} = \mathbf{K}(\mathbf{t})\gamma(\mathbf{t}) - \mathbf{c} = \mathbf{0} \quad (3.25)$$

V tem razdelku najprej podamo izrek o implicitni funkciji [lee2003smooth] in nato pokažemo, kako z njim argumentiramo obstoj gladkih funkcij γ , $\bar{\mathbf{a}}$ in $\bar{\mu}$.

Izrek 3.1 (Izrek o implicitni funkciji [lee2003smooth]). *Naj bo $f : \mathbb{R}^{m+n} \rightarrow \mathbb{R}^m$ zvezno odvedljiva funkcija in naj ima $\mathbb{R}^{m+n}$ koordinate $(\mathbf{x}, \mathbf{y})$.*

Fiksirajmo točko $(\mathbf{a}, \mathbf{b}) = (a_1, a_2, \dots, a_m, b_1, b_2, \dots, b_n)$, kjer velja $f(\mathbf{a}, \mathbf{b}) = \mathbf{0}$ in $\mathbf{0} \in \mathbb{R}^m$ je ničelni vektor. Če je Jacobijeva matrika:

$$J_{f,\mathbf{y}}(\mathbf{a}, \mathbf{b}) = \left[\frac{\partial f_i}{\partial y_j}(\mathbf{a}, \mathbf{b}) \right] \quad (3.26)$$

obrnjiva, potem obstaja odprta množica $U \in \mathbb{R}^n$, ki vsebuje $\mathbf{a}$, tako da obstaja enolična funkcija $g : U \rightarrow \mathbb{R}^m$, kjer $g(\mathbf{a}) = \mathbf{b}$ in $f(\mathbf{x}, g(\mathbf{x})) = \mathbf{0}$ za vse $\mathbf{x} \in U$. Nadalje, g je zvezno odvedljiva in Jacobijeva matrika parcialnih odvodov g v U je definirana kot:

$$\left[\frac{\partial g_i}{\partial x_j}(\mathbf{x}) \right]_{m \times n} = - [J_{f,\mathbf{y}}(\mathbf{x}, g(\mathbf{x}))]_{m \times m}^{-1} [J_{f,\mathbf{x}}(\mathbf{x}, g(\mathbf{x}))]_{m \times n} \quad (3.27)$$

, kjer je

$$J_{f,\mathbf{x}}(\mathbf{a}, \mathbf{b}) = \left[\frac{\partial f_i}{\partial x_j}(\mathbf{a}, \mathbf{b}) \right] \quad (3.28)$$

Izrek 3.2 (Gladkost rešitve KKT). *Naj velja enačba $\mathbf{K}\mathbf{y}^* - \mathbf{c} = \mathbf{0}$, kjer je $\mathbf{y}^* = (\mathbf{a}^*, \boldsymbol{\mu}^*)$ rešitev KKT QP z enakostnimi omejitvami. Potem obstaja odprta množica $U \subset \mathbb{R}^n$ in enolične gladke funkcije $\gamma : U \rightarrow \mathbb{R}^{m+n}$, $\bar{\mathbf{a}} : U \rightarrow \mathbb{R}^n$, $\bar{\mu} : U \rightarrow \mathbb{R}^m$, kjer velja $\gamma(\mathbf{t}) = (\bar{\mathbf{a}}(\mathbf{t}), \bar{\mu}(\mathbf{t})) = (\mathbf{a}^*, \boldsymbol{\mu}^*)$.*

Dokaz. Pokazati moramo, da je Jacobijeva matrika parcialnih odvodov enačbe $K\mathbf{y}^* - \mathbf{c}$ glede na $\mathbf{y}^*$ obrnljiva. Ni težko videti, da je Jacobijeva matrika parcialnih odvodov kar KKT matrika $\mathbf{K}$. Spomnimo, matrika $\mathbf{K}$ ima bločno strukturo in je definirana v enačbi (3.23). Ker je $\mathbf{Q}$ pozitivno definitna in ker ima $\mathbf{A}$ poln rang, je matrika $\mathbf{K}$ obrnljiva (dokaz za to je podan v [EPFL_Part5]). S tem je pogoj izrek o implicitni funkciji izpolnjen in sledi obstoj in enoličnost gladih funkcij γ , $\bar{\mathbf{a}}$ in $\bar{\mu}$. $\square$

3.4.3 Računanje gradienta

V tem razdelku predstavimo, kako izračunamo gradient cenovne funkcije $F(\mathbf{t})$. Začnemo z zapisom parcialnega odvoda $F(\mathbf{t})$ glede na komponento t_i :

$$\partial_{t_i} F = \bar{\mathbf{a}}^\top \mathbf{Q}(\partial_{t_i} \bar{\mathbf{a}}) + \frac{1}{2} \bar{\mathbf{a}}^\top (\partial_{t_i} \mathbf{Q}) \bar{\mathbf{a}} \quad (3.29)$$

Iz enačbe (3.25) lahko izrazimo parcialni odvod $\partial_{t_i} \gamma(\mathbf{t})$ kot:

$$\begin{aligned} \mathbf{K}\gamma(\mathbf{t}) - \mathbf{c} &= \mathbf{0} \\ \partial_{t_i}(\mathbf{K}\gamma(\mathbf{t}) - \mathbf{c}) &= \mathbf{0} \\ \partial_{t_i} \mathbf{K}\gamma(\mathbf{t}) + \mathbf{K}\partial_{t_i} \gamma(\mathbf{t}) &= \mathbf{0} \\ \partial_{t_i} \gamma(\mathbf{t}) &= -\mathbf{K}^{-1} \partial_{t_i} \mathbf{K}\gamma(\mathbf{t}) \end{aligned} \quad (3.30)$$

Ker je matrika $\mathbf{K}$ obrnljiva, lahko parcialni odvod $\partial_{t_i} \gamma(\mathbf{t})$ izračunamo eksplicitno, vendar v naši implementaciji inverza $\mathbf{K}^{-1}$ zaradi boljše časovne učinkovitosti pravzaprav nikoli ne izračunamo, vendar rešujemo sisteme enačb s to matriko.

Iz $\partial_{t_i} \gamma(\mathbf{t})$ lahko sedaj vzamemo komponento $\partial_{t_i} \bar{\mathbf{a}}(\mathbf{t})$ in jo vstavimo v izraz za $\partial_{t_i} F(\mathbf{t})$.

3.4.4 Računanje Hessove matrike

V tem razdelku predstavimo, kako izračunamo Hessovo matriko cenovne funkcije $F(\mathbf{t})$. Začnemo z zapisom parcialnega odvoda $\partial_{t_j} \partial_{t_i} F(\mathbf{t})$ glede na komponenti t_i in t_j , kjer uporabimo produktno pravilo za odvode:

$$\partial_{t_i t_j} F = \partial_{t_i} \bar{\mathbf{a}}^\top \mathbf{Q} \partial_{t_j} \bar{\mathbf{a}} + \bar{\mathbf{a}}^\top \partial_{t_i} \mathbf{Q} \partial_{t_j} \bar{\mathbf{a}} + \bar{\mathbf{a}}^\top \mathbf{Q} \partial_{t_i t_j} \bar{\mathbf{a}} + \bar{\mathbf{a}}^\top \partial_{t_j} \mathbf{Q} \partial_{t_i} \bar{\mathbf{a}} + \frac{1}{2} \bar{\mathbf{a}}^\top \partial_{t_i t_j} \mathbf{Q} \bar{\mathbf{a}} \quad (3.31)$$

Podobno kot pri izračunu gradienta, lahko izračunamo parcialni odvod $\partial_{t_j} \partial_{t_i} \gamma(\mathbf{t})$ tako, da rešimo spodnji sistem enačb:

$$\mathbf{K} \partial_{t_i t_j} \gamma = -(\partial_{t_i t_j} \mathbf{K} \gamma + \partial_{t_i} \mathbf{K} \partial_{t_j} \gamma + \partial_{t_j} \mathbf{K} \partial_{t_i} \gamma) \quad (3.32)$$

V tem podpoglavju smo pokazali, da lahko z uporabo KKT in izrek o implicitni funkciji izračunamo analitične parcialne odvode $\partial_{t_i} \bar{\mathbf{a}}(\mathbf{t})$ in $\partial_{t_j} \partial_{t_i} \bar{\mathbf{a}}(\mathbf{t})$, kar nam omogoča računanje analitičnega gradienta in Hessove matrike cenovne funkcije $F(\mathbf{t})$. S tem smo zagotovili vse potrebne gradnike, ki jih potrebujemo za optimizacijo $F(\mathbf{t})$. V naslednjem podpoglavju bomo predstavili optimizacijski algoritem, ki ga uporabljamo za minimizacijo $F(\mathbf{t})$, in sicer Newtonovo metodo z linijskim iskanjem z vračanjem.

3.5 Newtonova metoda in optimizacijski algoritem

V prejšnjih podpoglavjih smo predstavili, kako izračunati gradient in Hessovo matriko cenovne funkcije $F(\mathbf{t})$. V tem podpoglavju bomo predstavili iterativni optimizacijski algoritem, ki temelji na Newtonovi metodi, in najde lokalni minimum funkcije $F(\mathbf{t})$. Ker ima tudi zgornji problem dvonivojske strukture omejitvi $\mathbf{t} > \mathbf{0}$ in $\mathbf{1}^\top \mathbf{t} = T$, bomo gradient in Hessovo matriko projicirali na hiperravnino $\mathbf{1}^\top \mathbf{t} = T$ in nato uporabili linijsko iskanje z vračanjem, da zagotovimo, da so vektorji časov segmentov vedno pozitivni. V tem delu predpostavljamo trditev, da lokalni minimum funkcije $F(\mathbf{t})$ ne leži na robu množice $\mathbf{t} > \mathbf{0}$; za to predpostavko se sklicujemo na [sun2021fast], vendar priznavamo, da tudi v tem delu ta trditev ni formalno dokazana, niti podobnega dokaza nismo našli v literaturi. Vsekakor pa je ta trditev smiselna, saj bi lokalni minimum na robu množice $\mathbf{t} > \mathbf{0}$ pomenil, da je optimalna trajektorija sestavljena iz segmentov, katerih čas traja 0s.

3.5.1 Gradientni korak

Gradient projiciramo na hiperravnino $\mathbf{1}^\top \mathbf{t} = T$ tako, da od gradienta odštejemo komponento v smeri normalnega vektorja hiperravnine. Normalni vektor hiperravnine $\mathbf{1}^\top \mathbf{t} = T$ je $\mathbf{n} = \mathbf{1}$, zato lahko projicirani gradient izračunamo kot:

$$\nabla F_{proj}(\mathbf{t}) = \nabla F(\mathbf{t}) - \frac{\nabla F(\mathbf{t})^\top \mathbf{1}}{\mathbf{1}^\top \mathbf{1}} \mathbf{1} = \nabla F(\mathbf{t}) - \frac{1}{n} \sum_{i=1}^n \partial_{t_i} F(\mathbf{t}) = \nabla F(\mathbf{t}) - \nabla \bar{F}(\mathbf{t}) \quad (3.33)$$

3.5.2 Newtonov korak

Newtonov korak izračunamo tako, da rešimo sledeči KKT sistem:

$$\begin{bmatrix} \nabla^2 F(\mathbf{t}) & \mathbf{1} \\ \mathbf{1}^\top & 0 \end{bmatrix} \begin{bmatrix} \Delta \mathbf{t} \\ \lambda \end{bmatrix} = \begin{bmatrix} -\nabla F(\mathbf{t}) \\ 0 \end{bmatrix} \quad (3.34)$$

To je standarden pristop v Newtonovi metodi z enakostnimi omejitvami [nocedal2006numerical], kjer λ predstavlja Lagrangeevo multiplikatorje za omejitev $\mathbf{1}^\top \mathbf{t} = T$. Rešitev tega sistema nam da smer $\Delta \mathbf{t}$, v katero bomo naredili korak:

$$\mathbf{t}_{k+1} = \mathbf{t}_k + \alpha \Delta \mathbf{t}_k \quad (3.35)$$

3.5.3 Optimizacijski algoritem

V tem razdelku predstavimo celoten optimizacijski algoritem, ki ga uporabljamo za minimizacijo $F(\mathbf{t})$. Algoritem je predstavljen v algoritmu 2. V njem iterativno izračunavamo gradient in Hessovo matriko cenovne funkcije $F(\mathbf{t})$ z algoritmom 1. Uporabljamo hibridno linijsko iskanje z vračanjem, kjer se lahko izmenjujeta Newtonov in gradientni korak glede na to, kateri korak ponuja večje zmanjšanje funkcije $F(\mathbf{t})$; Predvsem v začetnih iteracijah je pogosto bolj učinkovit gradientni korak, v končnih iteracijah pa Newtonov korak. Pri linijskem iskanju z vračanjem za kriterij uporabljamo Armijo-Goldsteinov pogoj [nocedal2006numerical].

V implementaciji smo ta algoritem optimizirali tako, da smo s pomočjo vektORIZACIJE in vektorskih operacij, ki jih ponuja paket NumPy, izračunali vse parcialne odvode $\partial_{t_i} \bar{\mathbf{a}}$ in $\partial_{t_i t_j} \bar{\mathbf{a}}$ v enem klicu funkcije, namesto da bi zanko po komponentah izvajali ločeno. Natančnejši opis optimizacije algoritma je podan v dodatku B.

Algoritem 1 Izračun gradienta in Hesseve matrike (gradHess)

Require: Matrika $\mathbf{Q}$, matrika $\mathbf{A}$, optimalna rešitev $\mathbf{a}^*$, Lagrangeevi multiplikatorji $\boldsymbol{\mu}^*$, trenutni vektor časov $\mathbf{t}$

Ensure: Gradient $\nabla F(\mathbf{t})$ in Hesseva matrika $\nabla^2 F(\mathbf{t})$

```
1: Sestavi KKT matriko  $\mathbf{K}$  iz blokov  $\mathbf{Q}$ ,  $\mathbf{A}$ 
2: Sestavi vektor  $\gamma$  iz  $\mathbf{a}^*$  in  $\boldsymbol{\mu}^*$ 
3: Faktoriziraj  $\mathbf{K}$  s SuperLU
4: for  $i = 1$  to  $n$  do
5:   Izračunaj  $\partial_{t_i} \mathbf{Q}$  in  $\partial_{t_i} \mathbf{A}$ 
6:   Sestavi  $\partial_{t_i} \mathbf{K}$  iz blokov  $\partial_{t_i} \mathbf{Q}$  in  $\partial_{t_i} \mathbf{A}$ 
7:   Izračunaj desno stran:  $\mathbf{B}_i = -\partial_{t_i} \mathbf{K} \gamma$ 
8:   Reši linearni sistem:  $\mathbf{K} \partial_{t_i} \gamma = \mathbf{B}_i$ 
9:   Ekstrahiraj  $\partial_{t_i} \bar{\mathbf{a}}$  iz  $\partial_{t_i} \gamma$ 
10:  Izračunaj komponento gradienta  $\partial_{t_i} F$  z enačbo (3.29)
11: end for
12: for  $i = 1$  to  $n$  do
13:   for  $j = i$  to  $n$  do
14:     Izračunaj  $\partial_{t_i t_j} \mathbf{Q}$  in  $\partial_{t_i t_j} \mathbf{A}$ 
15:     Sestavi  $\partial_{t_i t_j} \mathbf{K}$  iz blokov  $\partial_{t_i t_j} \mathbf{Q}$  in  $\partial_{t_i t_j} \mathbf{A}$ 
16:     Izračunaj  $\mathbf{B}_{ij} = -(\partial_{t_i t_j} \mathbf{K} \gamma + \partial_{t_i} \mathbf{K} \partial_{t_j} \gamma + \partial_{t_j} \mathbf{K} \partial_{t_i} \gamma)$ 
17:     Reši linearni sistem:  $\mathbf{K} \partial_{t_i t_j} \gamma = \mathbf{B}_{ij}$ 
18:     Ekstrahiraj  $\partial_{t_i t_j} \bar{\mathbf{a}}$  iz  $\partial_{t_i t_j} \gamma$ 
19:     Izračunaj komponento Hesseve matrike  $\partial_{t_i t_j} F$  z enačbo (3.31)
20:   end for
21: end for
22: return  $\nabla F(\mathbf{t})$ ,  $\nabla^2 F(\mathbf{t})$ 
```

Algoritem 2 Newtonova metoda za optimizacijo časov segmentov

Require: Začetna točka $\mathbf{t}_0$, maksimalno število iteracij max_iter , toleranca ϵ

Ensure: Optimalna točka $\mathbf{t}^*$

```
1: for  $k = 0$  to  $K$  do
2:   Izračunaj cenovno matriko  $\mathbf{Q}(\mathbf{t}_k)$ 
3:   for  $d \in \{x, y, z\}$  do
4:     Izračunaj matriko  $\mathbf{A}_d(\mathbf{t}_k)$  in vektor  $\mathbf{b}_d(\mathbf{t}_k)$ 
5:     Reši QP:  $\min_{\mathbf{a}} \frac{1}{2} \mathbf{a}^\top \mathbf{Q}(\mathbf{t}_k) \mathbf{a}$  p.p.  $\mathbf{A}_d(\mathbf{t}_k) \mathbf{a} = \mathbf{b}_d(\mathbf{t}_k)$ 
6:     Dobi rešitev  $\mathbf{a}_d^*$  in Lagrangeeve multiplikatorje  $\boldsymbol{\mu}_d^*$ 
7:     Izračunaj  $\nabla F_d(\mathbf{t}_k), \nabla^2 F_d(\mathbf{t}_k)$  z algoritmom 1
8:   end for
9:   Izračunaj  $F(\mathbf{t}_k) = F_x(\mathbf{t}_k) + F_y(\mathbf{t}_k) + F_z(\mathbf{t}_k)$ 
10:  Izračunaj  $\nabla F(\mathbf{t}_k) = \nabla F_x(\mathbf{t}_k) + \nabla F_y(\mathbf{t}_k) + \nabla F_z(\mathbf{t}_k)$ 
11:  Izračunaj  $\nabla^2 F(\mathbf{t}_k) = \nabla^2 F_x(\mathbf{t}_k) + \nabla^2 F_y(\mathbf{t}_k) + \nabla^2 F_z(\mathbf{t}_k)$ 
12:  Izračunaj normaliziran projicirani gradientni korak  $\nabla \tilde{\mathbf{p}}(\mathbf{t}_k)$ .
13:  Reši KKT sistem (3.34) in izlušči Newtonov korak  $\mathbf{p}_k$ .
14:   $\alpha \leftarrow 1$ 
15:  while  $F(\mathbf{t}_k + \alpha \mathbf{p}_k) > F(\mathbf{t}_k) + c\alpha \nabla F(\mathbf{t}_k)^\top \mathbf{p}_k$  in  $F(\mathbf{t}_k + \alpha \tilde{\mathbf{p}}_k) > F(\mathbf{t}_k) + c\alpha \nabla F(\mathbf{t}_k)^\top \tilde{\mathbf{p}}_k$  (Armijo_Goldsteinov pogoj) do
16:     $\alpha \leftarrow \tau \alpha$ 
17:  end while
18:  if  $F(\mathbf{t}_k + \alpha \mathbf{p}_k) < F(\mathbf{t}_k + \alpha \tilde{\mathbf{p}}_k)$  then
19:    Posodobi  $\mathbf{t}_{k+1} = \mathbf{t}_k + \alpha \mathbf{p}_k$ 
20:  else
21:    Posodobi  $\mathbf{t}_{k+1} = \mathbf{t}_k + \alpha \tilde{\mathbf{p}}_k$ 
22:  end if
23: end for
24: return  $\mathbf{t}_k$ 
```

3.6 Pohitritve

V prejšnjih podpoglavjih smo predstavili metodo za izračun analitičnega gradienta in Hessove matrike cenovne funkcije $F(\mathbf{t})$, kar nam omogoča uporabo Newtonove metode z linijskim iskanjem z vračanjem (algoritem 2) za minimizacijo $F(\mathbf{t})$. V tem podpoglavju se bomo posvetili podrobnostim implementacije, ki so ključne za doseganje visoke časovne učinkovitosti naše metode. Najprej predstavimo, kako faktoriziramo matriko $\mathbf{K}$, kjer izkoristimo njeno redko strukturo. Nato opišemo metodo izračuna desne strani linearnih sistemov enačb pri računanju prvih in drugih parcialnih odvodov F . Na koncu predstavimo pasovno strukturo Hessove matrike, ki nam omogoča dodatno pohitritev optimizacijskega algoritma.

3.6.1 Faktorizacija KKT matrike $\mathbf{K}$

Kot opisano v enačbi (3.23), je KKT matrika definirana kot:

$$\mathbf{K} = \begin{bmatrix} \mathbf{Q} & \mathbf{A}^\top \\ \mathbf{A} & \mathbf{0} \end{bmatrix} \quad (3.36)$$

V poglavju 3.2 smo pokazali, da je matrika $\mathbf{Q}$ bločno diagonalna matrika, matrika $\mathbf{A}$ pa je bločno bidiagonalna matrika (ima neničelne bloke na glavni diagonali in prvi poddiagonali). Ker ima $\mathbf{Q}$ n , $\mathbf{A}$ pa $2n$ neničelnih blokov, je torej matrika $\mathbf{K}$ redka matrika z $O(n)$ neničelnimi elementi (natančneje, ima $5n$ neničelnih blokov), kjer je n število segmentov trajektorije. Zaradi tega je smiselno uporabiti metodo faktorizacije, ki je prilagojena za redke matrike, kot je SuperLU [[superlu1997superlu](#)], namesto da bi izračunali inverz matrike $\mathbf{K}$, kar bi bilo računsko bolj zahtevno.

3.6.2 Računanje desne strani linearnih sistemov enačb

V poglavjih 3.4.3 in 3.4.4 smo pokazali, da za izračun analitičnega gradienta in Hessove matrike cenovne funkcije $F(\mathbf{t})$ potrebujemo rešiti linearna sistema enačb z matriko $\mathbf{K}$ in desno stranjo, ki vključuje parcialne odvodne matrike $\partial_{t_i}\mathbf{K}$ in $\partial_{t_it_j}\mathbf{K}$. V tem podpoglavju bomo predstavili, kako učinkovito izračunati te desne strani, pri čemer bomo izkoristili strukturo matrike $\mathbf{K}$ in njene parcialne odvodne matrike.

Naš glavni cilj je učinkovito izračunati desne strani linearnih sistemov enačb, ki jih potrebujemo za izračun gradienta in Hessove matrike. Za izračun gradienta potrebujemo izraz:

$$\mathbf{B}_i = -\partial_{t_i}\mathbf{K}\gamma, \quad (3.37)$$

kjer $\mathbf{B}_i$ predstavlja desno stran linearnega sistema enačb, ki ga moramo rešiti za izračun $\partial_{t_i}\gamma$ in $i \in 1, \dots, n$:

$$\mathbf{K}\partial_{t_i}\gamma = \mathbf{B}_i \quad (3.38)$$

Za izračun Hessove matrike pa potrebujemo izraz:

$$\mathbf{B}_{ij} = -(\partial_{t_it_j}\mathbf{K}\gamma + \partial_{t_i}\mathbf{K}\partial_{t_j}\gamma + \partial_{t_j}\mathbf{K}\partial_{t_i}\gamma), \quad (3.39)$$

kjer $\mathbf{B}_{ij}$ predstavlja desno stran linearnega sistema enačb, ki ga moramo rešiti za izračun $\partial_{t_it_j}\gamma$ in $i, j \in 1, \dots, n$:

$$\mathbf{K}\partial_{t_it_j}\gamma = \mathbf{B}_{ij} \quad (3.40)$$

Najprej se osredotočimo na učinkovit izračun parcialnih odvodov prvega in drugega reda matrike $\mathbf{K}$. Iz spodnjih enačb

$$\begin{aligned}\partial_{t_i}\mathbf{K} &= \begin{bmatrix} \partial_{t_i}\mathbf{Q} & \partial_{t_i}\mathbf{A}^\top \\ \partial_{t_i}\mathbf{A} & \mathbf{0} \end{bmatrix}, \\ \partial_{t_it_j}\mathbf{K} &= \begin{bmatrix} \partial_{t_it_j}\mathbf{Q} & \partial_{t_it_j}\mathbf{A}^\top \\ \partial_{t_it_j}\mathbf{A} & \mathbf{0} \end{bmatrix}\end{aligned}\quad (3.41)$$

lahko vidimo, da moramo izračunati parcialne odvodne matrike $\partial_{t_i}\mathbf{Q}$, $\partial_{t_i}\mathbf{A}$, $\partial_{t_it_j}\mathbf{Q}$ in $\partial_{t_it_j}\mathbf{A}$. Vsako od njih obravnavamo posamezno.

- $\partial_{t_i}\mathbf{Q}$: Spomnimo, matrika $\mathbf{Q}$ ima sledečo strukturo:

$$\mathbf{Q}(\mathbf{t}) = \begin{bmatrix} \mathbf{Q}_1(t_1) & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & \mathbf{Q}_2(t_2) & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & \mathbf{Q}_n(t_n) \end{bmatrix} \quad (3.42)$$

Vidimo lahko, da je le i -ti blok na diagonalni odvisen od t_i , zato je $\partial_{t_i}\mathbf{Q}$ matrika, kjer je le i -ti blok na diagonalni neničeln in je enak $\partial_{t_i}\mathbf{Q}_i(t_i)$, medtem ko so vsi ostali bloki ničelni:

$$\partial_{t_i}\mathbf{Q} = \text{diag}(\mathbf{0}, \dots, \mathbf{0}, \partial_{t_i}\mathbf{Q}_i, \mathbf{0}, \dots, \mathbf{0}) \quad (3.43)$$

- $\partial_{t_i}\mathbf{A}$: Spomnimo, matrika $\mathbf{A}$ ima sledečo strukturo:

$$\mathbf{A}(\mathbf{t}) = \begin{bmatrix} \mathbf{A}_1(0) & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{0} \\ \mathbf{A}_1(t_1) & -\mathbf{A}_2(0) & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{A}_2(t_2) & -\mathbf{A}_3(0) & \cdots & \mathbf{0} & \mathbf{0} \\ \vdots & \vdots & \ddots & \ddots & \vdots & \vdots \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{A}_{n-1}(t_{n-1}) & \mathbf{A}_n(0) \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{A}_n(t_n) \end{bmatrix} \quad (3.44)$$

Vidimo lahko, da so bloki po zgornji diagonalni neodvisni od $\mathbf{t}$, zato so njihovi parcialni odvodi ničelni. Bloki po spodnji diagonalni so odvisni od t_i za $i = 1, \dots, n-1$, zato je $\partial_{t_i}\mathbf{A}$ matrika, kjer so le bloki po spodnji diagonalni neničelni in so enaki $\partial_{t_i}\mathbf{A}_i(t_i)$, medtem ko so vsi ostali bloki ničelni:

$$\partial_{t_i}\mathbf{A} = \begin{bmatrix} \mathbf{0} & \mathbf{0} & \cdots & \cdots & \cdots & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \cdots & \cdots & \cdots & \mathbf{0} & \mathbf{0} \\ \vdots & \vdots & \ddots & \ddots & \vdots & \vdots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & \partial_{t_i}\mathbf{A}_i & \mathbf{0} & \cdots & \mathbf{0} \\ \vdots & \vdots & & \vdots & \ddots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{0} \end{bmatrix} \quad (3.45)$$

- $\partial_{t_it_j}\mathbf{Q}$ lahko izračunamo, da $\partial_{t_i}\mathbf{Q}$ odvajamo po t_j . Opazimo lahko, da, če je $i \neq j$, potem je $\partial_{t_it_j}\mathbf{Q}$ ničelna matrika. Sicer pa velja:

$$\partial_{t_it_i}\mathbf{Q} = \text{diag}(\mathbf{0}, \dots, \mathbf{0}, \partial_{t_it_i}\mathbf{Q}_i, \mathbf{0}, \dots, \mathbf{0}) \quad (3.46)$$

- $\partial_{t_i t_j} \mathbf{A}$ analogno lahko izračunamo, da $\partial_{t_i} \mathbf{A}$ odvajamo po t_j . Opazimo lahko, da, če je $i \neq j$, potem je $\partial_{t_i t_j} \mathbf{A}$ ničelna matrika. Sicer pa velja:

$$\partial_{t_i t_i} \mathbf{A} = \begin{bmatrix} \mathbf{0} & \mathbf{0} & \cdots & \cdots & \cdots & \mathbf{0} \\ \vdots & \ddots & \ddots & & & \vdots \\ \mathbf{0} & \cdots & \partial_{t_i t_i} \mathbf{A}_i & \mathbf{0} & \cdots & \mathbf{0} \\ \vdots & & & \ddots & \ddots & \vdots \\ \mathbf{0} & \cdots & \cdots & \cdots & \mathbf{0} & \mathbf{0} \end{bmatrix} \quad (3.47)$$

Pokazali smo, da so parcialni odvodi matrice $\mathbf{K}$ redki, imajo pa tudi bločno strukturo, in namreč imajo največ tri neničelne bloke, katerih velikost je odvisna od stopnje polinoma d in zelene gladkosti trajektorije k , ne pa tudi od števila segmentov n . Lahko torej trdimo, da je $nnz(\partial_{t_i} \mathbf{K})$ in $nnz(\partial_{t_i t_j} \mathbf{K})$ $O(1)$ v odvisnosti od števila segmentov n , kjer nnz predstavlja število neničelnih (*angl. non-zero*) elementov matrice.

Sedaj lahko izračunamo desno stran linearnih sistemov enačb tako, da izračunamo le neničelne elemente vektorja $\mathbf{B}_i$ in $\mathbf{B}_{ij}$. Pri B_i

$$B_i = - \begin{bmatrix} \partial_{t_i} \mathbf{Q} & \partial_{t_i} \mathbf{A}^\top \\ \partial_{t_i} \mathbf{A} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \bar{\mathbf{a}} \\ \bar{\mu} \end{bmatrix} = - \begin{bmatrix} \partial_{t_i} \mathbf{Q} \bar{\mathbf{a}} + \partial_{t_i} \mathbf{A}^\top \bar{\mu} \\ \partial_{t_i} \mathbf{A} \bar{\mathbf{a}} \end{bmatrix} = - \begin{bmatrix} \mathbf{0} \\ \vdots \\ \partial_{t_i} \mathbf{Q}_i \bar{\mathbf{a}}_i + \partial_{t_i} \mathbf{A}_i^\top \bar{\mu}_i \\ \vdots \\ \mathbf{0} \\ \mathbf{0} \\ \vdots \\ \partial_{t_i} \mathbf{A}_i \bar{\mathbf{a}}_i \\ \vdots \\ \mathbf{0} \end{bmatrix} \quad (3.48)$$

$$\begin{aligned} B_{ij} &= - (\partial_{t_i t_j} \mathbf{K} \gamma + \partial_{t_i} \mathbf{K} \partial_{t_j} \gamma + \partial_{t_j} \mathbf{K} \partial_{t_i} \gamma) = \\ &= - \left(\begin{bmatrix} \partial_{t_i t_j} \mathbf{Q} & \partial_{t_i t_j} \mathbf{A}^\top \\ \partial_{t_i t_j} \mathbf{A} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \bar{\mathbf{a}} \\ \bar{\mu} \end{bmatrix} + \begin{bmatrix} \partial_{t_i} \mathbf{Q} & \partial_{t_i} \mathbf{A}^\top \\ \partial_{t_i} \mathbf{A} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \partial_{t_j} \bar{\mathbf{a}} \\ \partial_{t_j} \bar{\mu} \end{bmatrix} + \begin{bmatrix} \mathbf{Q} & \mathbf{A}^\top \\ \mathbf{A} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \partial_{t_i t_j} \bar{\mathbf{a}} \\ \partial_{t_i t_j} \bar{\mu} \end{bmatrix} \right) = \\ &= - \begin{bmatrix} \partial_{t_i t_j} \mathbf{Q} \bar{\mathbf{a}} + \partial_{t_i} \mathbf{Q} \partial_{t_j} \bar{\mathbf{a}} + \partial_{t_j} \mathbf{Q} \partial_{t_i} \bar{\mathbf{a}} + \partial_{t_i t_j} \mathbf{A}^\top \bar{\mu} + \partial_{t_i} \mathbf{A}^\top \partial_{t_j} \bar{\mu} + \partial_{t_j} \mathbf{A}^\top \partial_{t_i} \bar{\mu} \\ \partial_{t_i t_j} \mathbf{A} \bar{\mathbf{a}} + \partial_{t_i} \mathbf{A} \partial_{t_j} \bar{\mathbf{a}} + \partial_{t_j} \mathbf{A} \partial_{t_i} \bar{\mathbf{a}} \end{bmatrix} \end{aligned} \quad (3.49)$$

V primeru $i = j$ lahko izračunamo B_{ii} kot:

$$\begin{aligned}
B_{ii} &= - \begin{bmatrix} \partial_{t_i t_i} \mathbf{Q} \bar{\mathbf{a}} + 2\partial_{t_i} \mathbf{Q} \partial_{t_i} \bar{\mathbf{a}} + \partial_{t_i t_i} \mathbf{A}^\top \mu + 2\partial_{t_i} \mathbf{A}^\top \partial_{t_i} \mu \\ \partial_{t_i t_i} \mathbf{A} \bar{\mathbf{a}} + 2\partial_{t_i} \mathbf{A} \partial_{t_i} \bar{\mathbf{a}} \end{bmatrix} \\
&= - \begin{bmatrix} \mathbf{0} \\ \vdots \\ \partial_{t_i t_i} \mathbf{Q}_i \bar{\mathbf{a}}_i + 2\partial_{t_i} \mathbf{Q}_i \partial_{t_i} \bar{\mathbf{a}}_i + \partial_{t_i t_i} \mathbf{A}_i^\top \bar{\mu}_i + 2\partial_{t_i} \mathbf{A}_i^\top \partial_{t_i} \bar{\mu}_i \\ \vdots \\ \mathbf{0} \\ \mathbf{0} \\ \vdots \\ \partial_{t_i t_i} \mathbf{A}_i \bar{\mathbf{a}}_i + 2\partial_{t_i} \mathbf{A}_i \partial_{t_i} \bar{\mathbf{a}}_i \\ \vdots \\ \mathbf{0} \end{bmatrix} \quad (3.50)
\end{aligned}$$

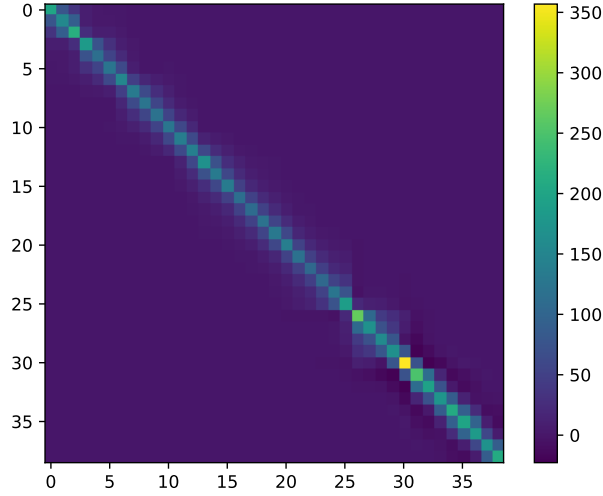
V primeru $i \neq j$ pa člena s $\partial_{t_i t_j} \mathbf{Q}$ in $\partial_{t_i t_j} \mathbf{A}$ izgineta, zato lahko B_{ij} izračunamo kot:

$$\begin{aligned}
B_{ij} = B_{ji} &= - \begin{bmatrix} \partial_{t_i} \mathbf{Q} \partial_{t_j} \bar{\mathbf{a}} + \partial_{t_j} \mathbf{Q} \partial_{t_i} \bar{\mathbf{a}} + \partial_{t_i} \mathbf{A}^\top \partial_{t_j} \mu + \partial_{t_j} \mathbf{A}^\top \partial_{t_i} \mu \\ \partial_{t_i} \mathbf{A} \partial_{t_j} \bar{\mathbf{a}} + \partial_{t_j} \mathbf{A} \partial_{t_i} \bar{\mathbf{a}} \end{bmatrix} \\
&= - \begin{bmatrix} \mathbf{0} \\ \vdots \\ \partial_{t_i} \mathbf{Q}_i \partial_{t_j} \bar{\mathbf{a}}_i + \partial_{t_j} \mathbf{Q}_i \partial_{t_i} \bar{\mathbf{a}}_i + \partial_{t_i} \mathbf{A}_i^\top \partial_{t_j} \bar{\mu}_i \\ \vdots \\ \partial_{t_j} \mathbf{Q}_j \partial_{t_i} \bar{\mathbf{a}}_j + \partial_{t_i} \mathbf{Q}_j \partial_{t_j} \bar{\mathbf{a}}_j + \partial_{t_j} \mathbf{A}_j^\top \partial_{t_i} \bar{\mu}_j \\ \vdots \\ \mathbf{0} \\ \mathbf{0} \\ \vdots \\ \partial_{t_i} \mathbf{A}_i \partial_{t_j} \bar{\mathbf{a}}_i \\ \vdots \\ \partial_{t_j} \mathbf{A}_j \partial_{t_i} \bar{\mathbf{a}}_j \\ \vdots \\ \mathbf{0} \end{bmatrix} ; \quad (3.51)
\end{aligned}$$

Vidimo lahko, da sama konstrukcija vektorjev $\mathbf{B}_i$ in $\mathbf{B}_{ij}$ zahteva $O(1)$ operacij glede na n . V dodatku B pokažemo, kako v primeru gradienta lahko vektoriziramo opisani proces in namesto računanja v zanki po komponentah izračunamo vse $\mathbf{B}_i$ naenkrat s pomočjo tenzorjev. Enako lahko storimo tudi za Hessovo matriko, vendar v tem primeru bralca napotimo na implementacijo.

3.6.3 Pasovna struktura Hessove matrike

Newtonova metoda v splošnem zahteva izračun in faktorizacijo Hessove matrike, kar je lahko za probleme z veliko spremenljivkami računsko zahtevno; Če računamo



Slika 3: Pasovna struktura Hessove matrike za $n = 39$ segmentov trajektorije.

vsak parcialni odvod drugega reda cenovne funkcije $F(\mathbf{t})$, potem je število parcialnih odvodov drugega reda $O(n^2)$, kjer n je število segmentov trajektorije. V našem primeru pa se izkaže, da s tem, da če izračunamo samo diagonalo in nekaj pasov okoli nje, dosežemo zelo dober približek Hessove matrike, s tem pa tudi znižamo časovno zahtevnost na $O(n)$. Na sliki 3 je prikazana Hessova matrika na primeru z $n = 39$ segmenti trajektorije. Vidimo lahko, da so neničelni elementi matrike skoncentrirani v pasovih okoli diagonale. V podpoglavju 4.3 bomo pokazali, da je uporaba samo diagonale in 4 pasov pod in nad diagonalo dovolj za doseganje dobre konvergence optimizacijskega algoritma, hkrati pa nam omogoči pohitritev algoritma.

4 Eksperimenti

V tem poglavju predstavimo rezultate eksperimentov, ki smo jih izvedli za oceno učinkovitosti naše metode. Najprej predstavimo implementacijske podrobnosti in uporabljene knjižnice. Nato analiziramo konvergenco naše metode, ki jo primerjamo z dvema drugima metodama, in sicer z metodo, ki uporablja samo analitično izračunani gradient, in metodo trust-region, ki uporablja numerično izračunani gradient in Hessovo matriko. Poleg tega analiziramo hitrost naše metode in njeno odvisnost od števila segmentov trajektorije. Na koncu proučimo pasovno strukturo Hessove matrike, kjer nas zanima kvaliteta algoritma glede na število pasov Hessove matrike, ki jih upoštevamo pri optimizaciji.

4.1 Implementacija in uporaba zunanjih knjižnic

Naša metoda je implementirana v programskem jeziku Python in je dostopna na povezavi github.com/maticpokorn/trajectory_optimization. V implementaciji se često zanašamo na operacije z redkimi matrikami za doseganje višje hitrosti in

zato uprabljamo modul `scipy.sparse` knjižnice SciPy [2020SciPy-NMeth]. Konkretno se poslužujemo metode SuperLU za faktorizacijo matrike v enačbi (3.23) [superlu1997superlu]. Za reševanje QP uporabljamo knjižnico OSQP [osqp]. Implementacijo naše metode sta omogočili še knjižnici NumPy [harris2020array] in Matplotlib [Hunter:2007].

Primer uporabe Tukaj prikažemo preprost primer uporabe naše metode v Python-u. Spodaj je prikazana koda, kjer definiramo problem z 11 vmesnimi točkami in skupnim časom trajektorije $T = 10$. Nato pokličemo metodo, ki implementira algoritem 2. V vrstici `solver = ...` določimo stopnjo polinoma, kateri odvod želimo optimizirati, stopnjo gladkosti trajektorije in dimenzionalnost prostora, v katerem se nahajajo točke.

```
from solver import NoTimestampSolver

solver = NoTimestampSolver(d=9, r=4, q=4, dimensions=3)

waypoints = [[0, 0, 0], [8, 4, 0], [2, 10, 0], [12, 14, 0],
              [8, 22, 0], [3, 20, 0], [0, 25, 0], [-8, 20, 0],
              [-4, 12, 0], [0, 8, 0], [4, 4, 0], [0, 0, 0]]

T = 10

result = solver.solve_backtracking_line_search(waypoints, T)
```

Vizualizacija dobljene trajektorije je prikazana na sliki 4, kjer so vidni tudi rezultati vmesnih iteracij optimizacije.

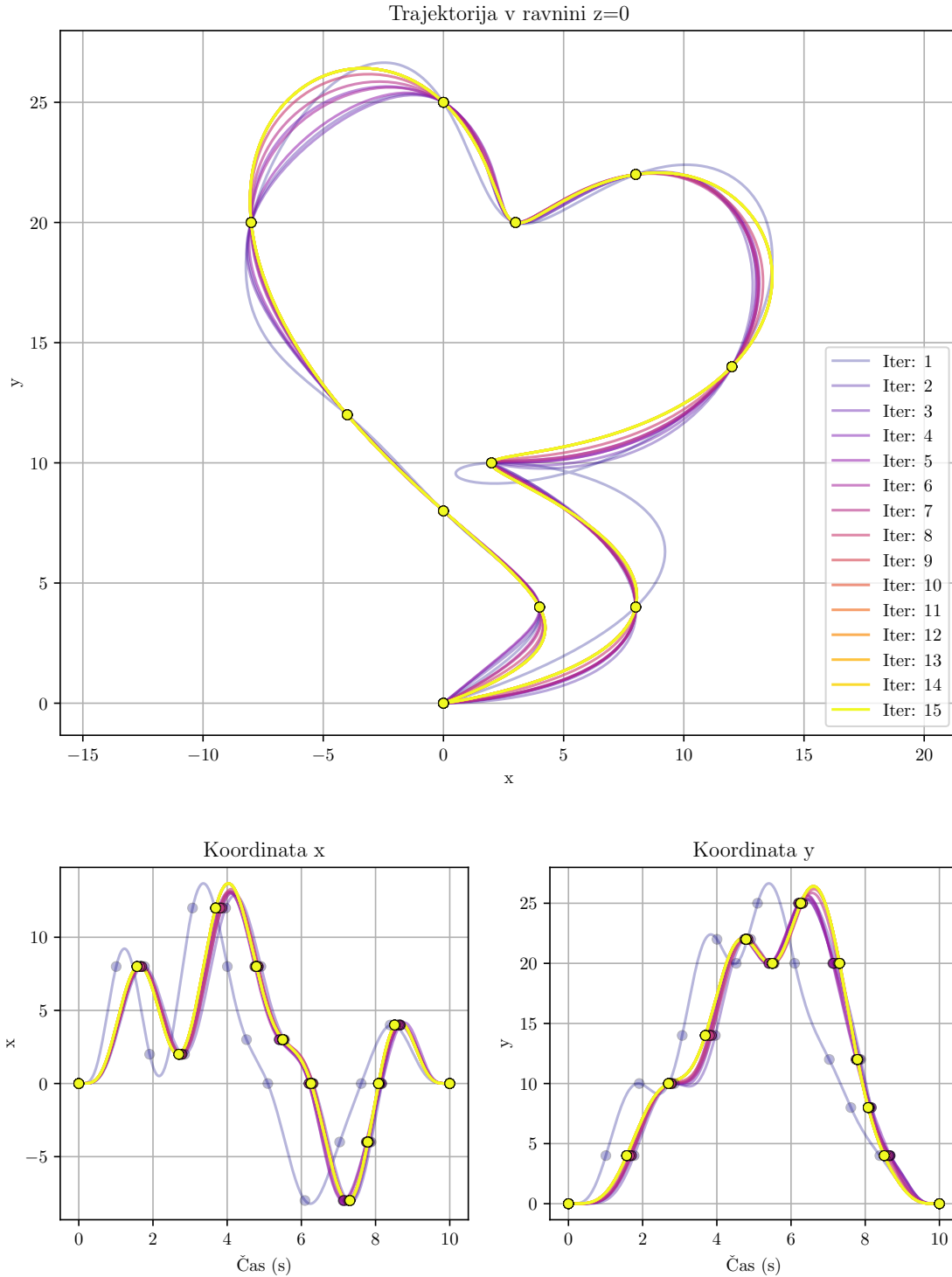
4.2 Konvergenca

V tem podpoglavju predstavimo rezultate ekseperimentov, s katerimi pokažemo primerjavo konvergence naše metode. Izmerimo konvergenco treh metod:

1. Newtonova metoda, ki uporablja analitično izračunani gradient in Hessovo matriko, kot je opisano v algoritmu 2.
2. Metoda gradientnega spusta, ki uporablja samo analitično izračunani gradient, ki ga projiciramo na hiperravnino $\mathbf{1}^\top \mathbf{t} = T$.
3. Dve metodi trust-constr (TC) iz paketa SciPy, ki uporabljata numerično oz. analitično izračunani gradient in Hessovo matriko ter metodo območja zaupanja [conn2000trust], ki dopušča linearne omejitve.

Za eksperimente uporabimo naključno generirane probleme, ki jih definiramo na dva načina:

- **Naključne točke:** Za vsak problem naključno generiramo n točk znotraj kocke $[0, 1]^3$.



Slika 4: Vizualizacija konvergence na primeru. Na zgornjem grafu je prikazana trajektorija in vmesne točke v ravnini $z = 0$ v različnih iteracijah optimizacije. Algoritem na vsakem koraku reši QP, ki je parametriziran s časovnim vektorjem $\mathbf{t}$. Na spodnjih grafih je prikazana trajektorija po komponentah x in y v odvisnosti od časa, kjer pike označujejo čas in mesto, ko trajektorija prečka vmesno točko.

- **Naključna premica:** Za vsak problem naključno generiramo n točk tako, da so koordinate i -te točke naključno izbrane znotraj kocke $[i - 1, i] \times [0, 1] \times [0, 1]$. Torej, točke si v grobem sledijo po premici od $[0, 0, 0]$ do $[n - 1, 0, 0]$.

Metodo poženemo tudi na različnem številu točk, in namreč si izberemo primere s 5, 10, 20, 30 in 40 točkami.

Začetna točka $\mathbf{t}_0$ je v vseh primerih izbrana na sledeči način:

$$t_{0i} = \frac{\|P_{i+1} - P_i\|}{\sum_{j=0}^{n-1} \|P_{j+1} - P_j\|} n \quad (4.1)$$

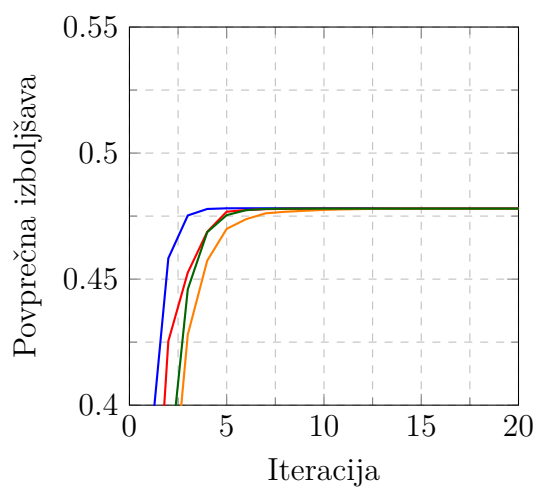
, torej čas segmenta je proporcionalen glede na evklidsko razdaljo med končnima točkama segmenta. Skupen čas trajektorije T je nastavljen na n .

Rezultati so prikazani na slikah 5 in 6. Na grafih je na x osi število iteracij optimizacije, na y osi pa povprečna izboljšava vrednosti cenovne funkcije $F(\mathbf{t}_i)$ glede na začetno vrednost $F(\mathbf{t}_0)$, izračunana s formulo $y_i = 1 - \frac{F(\mathbf{t}_i)}{F(\mathbf{t}_0)}$. Vsi rezultati pokažejo, da Newtonova metoda konvergira hitreje kot metoda, ki uporablja samo gradient in metoda trust-region, kar potrjuje učinkovitost naše metode. Poleg tega lahko opazimo, da se hitrost konvergence zmanjšuje z večjim številom segmentov trajektorije in Newtonova metoda večinoma izgubi prednost pred gradientnim spustom. Analitični in numerični TC sta večinoma slabša od ostalih dveh metod, čeprav analitični TC skonvergira k malo boljši rešitvi kot gradientni spust. V vseh primerih pa tudi velja, da Newtonova metoda konvergira k boljši rešitvi kot gradientni spust, vendar pa je ta razlika v končni vrednosti cenovne funkcije zelo majhna in v praksi ne bi bila opazna. Opazimo lahko, da pri prvem naboru naključnih primerov analitični TC konvergira k rešitvi bistveno hitreje kot numerični TC, kar je verjetno pokazatelj nenatančnosti numerične ocene odvodov, ki jih numerični TC uporablja. Pri drugem naboru pa se izkaže, da je numerični TC v začetnih iteracijah boljši kot analitični TC, kar bi lahko bila posledica tega, da F pri vrednostih $\mathbf{t}$ v začetnih iteracijah z večjo verjetnostjo ni konveksna; v vseh primerih namreč analitični TC skonvergira k boljši rešitvi kot numerični TC.

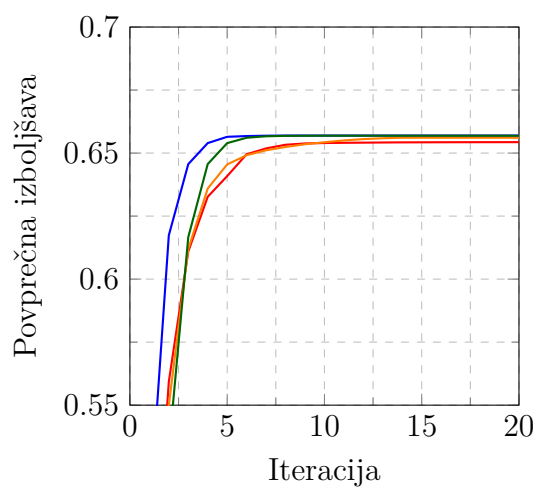
4.3 Pasovna struktura Hessove matrike

V podpoglavju 3.6.3 smo pokazali, da ima Hessova matrika strukturo, kjer so neničelni elementi skoncentrirani na diagonali in nekaj pasovih okoli nje, kar pomeni, da nam elementov matrike izven teh pasov ni treba izračunati, a še vedno dobimo dober približek Hessove matrike. Na sliki 7 je prikazana primerjava polne Hessove matrike in aproksimacij z različnim številom pasov okoli diagonale.

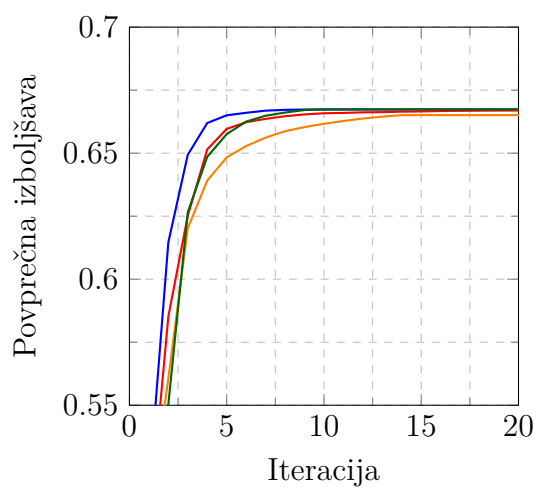
V tem podpoglavju podrobneje analiziramo vpliv števila izračunanih pasov Hessove matrike na konvergenco optimizacijskega algoritma. Pri tem opazujemo tako povprečen čas izračuna kot tudi število iteracij, potrebnih za konvergenco algoritma. Analizo izvedemo na naključnih primerih z 20, 30 in 40 vmesnimi točkami, zanima pa nas, kako se spreminjata konvergenca in čas izračuna glede na število pasov Hessove matrike, ki jih izračunamo. Rezultati so prikazani na slikah 8, 9 in 10. Na levem grafu lahko tako, kot v prejšnjem podpoglavju vidimo povprečno izboljšavo cenovne funkcije glede na iteracijo. Na srednjem grafu lahko vidimo celoten povprečen čas v sekundah, ki ga algoritem potrebuje do konvergence, na desnem grafu



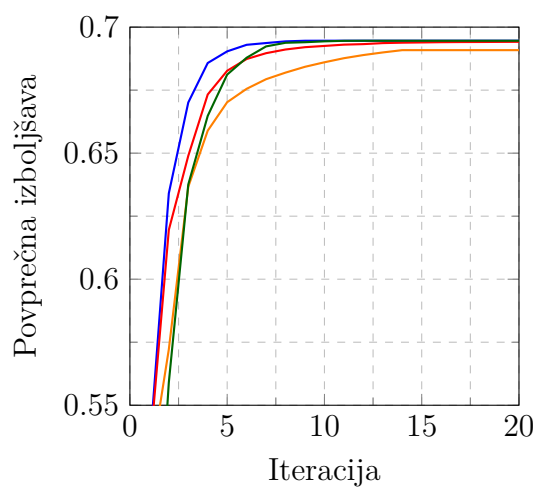
(a) 5 točk



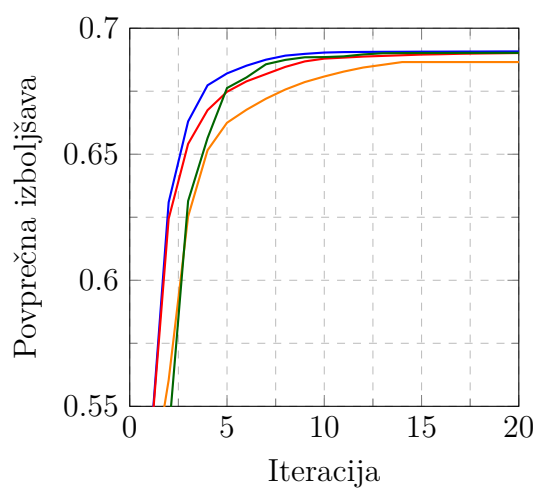
(b) 10 točk



(c) 20 točk



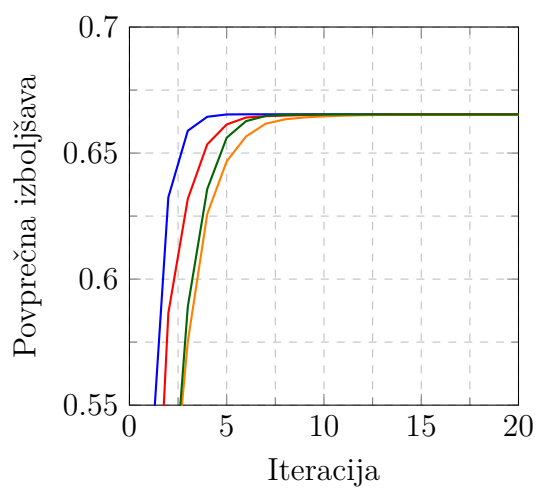
(d) 30 točk



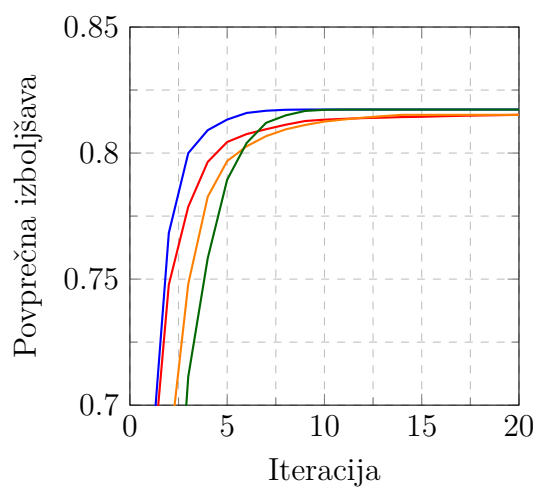
(e) 40 točk

— Newton
 — Gradient
 — Numerični TC
 — Analitični TC

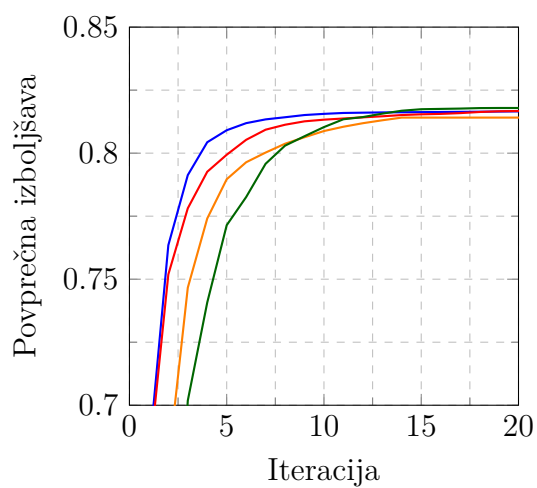
Slika 5: Konvergenca z naključnimi začetnimi točkami. V vseh primerih Newtonova metoda konvergira hitreje kot metoda z gradientnim spustom.



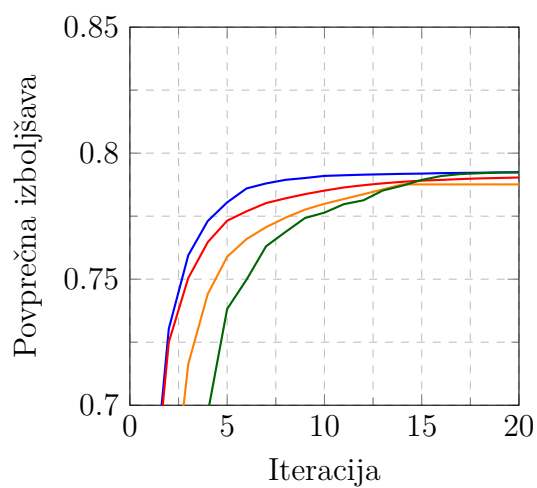
(a) 5 točk



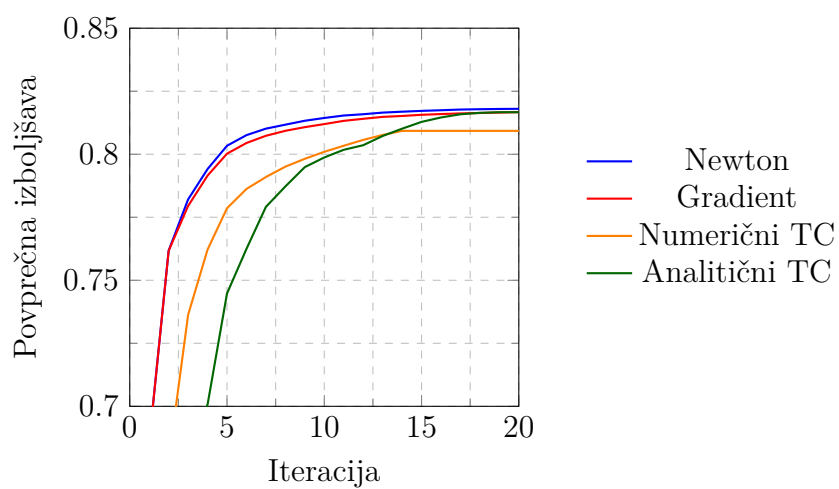
(b) 10 točk



(c) 20 točk

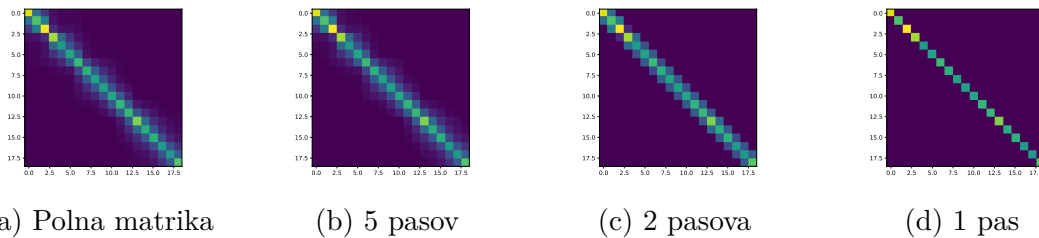


(d) 30 točk

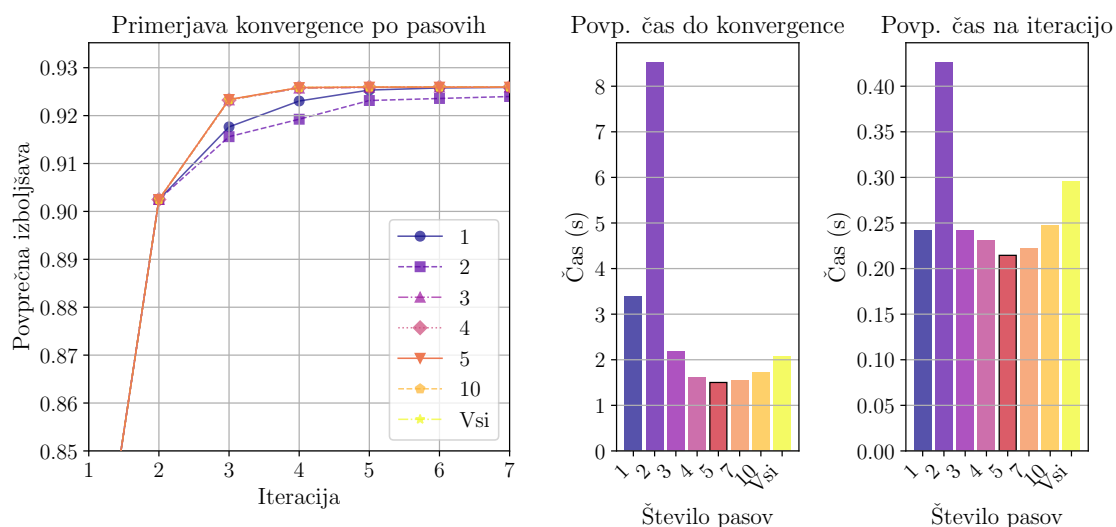


(e) 40 točk

Slika 6: Konvergenca z naključnimi točkami in premicami. V vseh primerih Newtonova metoda konvergira hitreje kot metoda z gradientnim spustom.



Slika 7: Primerjava polne Hesseve matrike in aproksimacij z različnim številom pasov okoli diagonale.

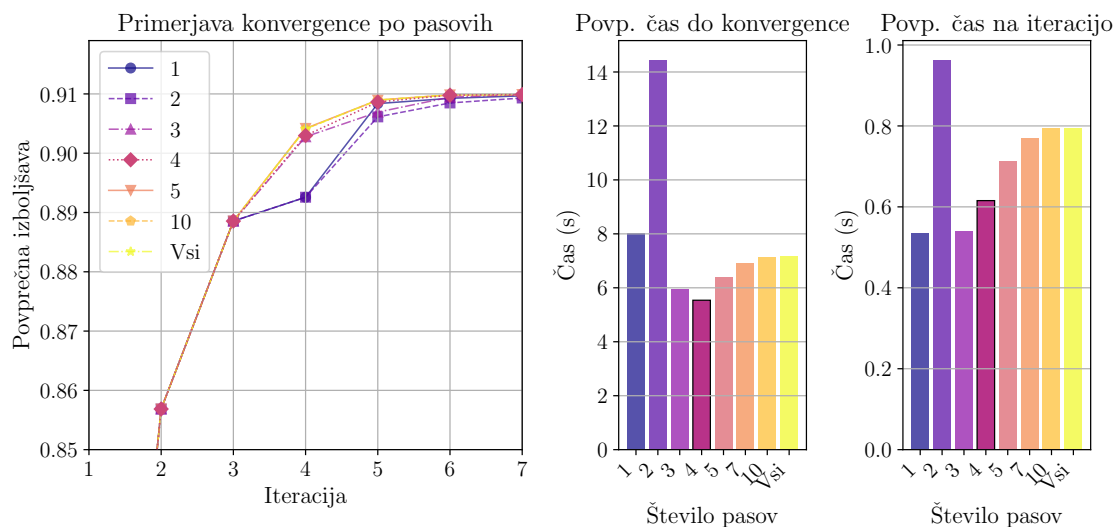


Slika 8: Vpliv števila izračunanih pasov Hesseve matrike na konvergenco za 20 vmesnih točk. Vidimo lahko, da je 5 pasov v tem primeru optimalno za najhitrejši izračun.

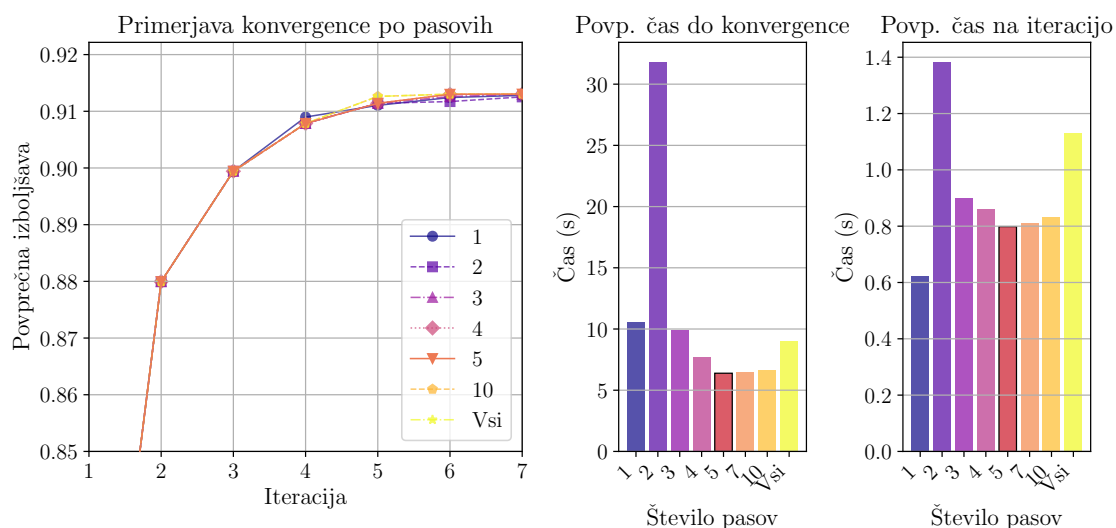
pa povprečen čas, ki ga potrebuje, da izvede eno iteracijo. Vidimo lahko, da je v vseh primerih optimalno število pasov okoli 4 ali 5, kar nam omogoča pohitritev algoritma brez večje izgube v hitrosti konvergence. Pri primerih s 30 in 40 segmenti se vidi, da je v primeru 1 pasu čas na iteracijo nižji kot v primeru s 4 ali 5 pasovi, a je število iteracij, potrebnih za konvergenco, večje zaradi slabega približka Hesseve matrike, kar vodi do večjega skupnega časa izračuna. V primeru z 2 pasovoma pa je v vseh primerih tako povprečen čas do konvergence kot povprečen čas na iteracijo bistveno višji od vseh ostalih.

4.4 Hitrost metode v realnem času

Poleg števila iteracij je za uporabnost metode pomembna tudi učinkovitost v realnem času. Čeprav je Newtonova metoda hitrejša od gradientnega spusta v smislu števila iteracij, je vsaka iteracija računsko zahtevnejša zaradi izračuna Hesseve matrike. Sliki 11 in 12 prikazujeta konvergenco v odvisnosti od porabljenega časa (angl. *wall clock time*) za naključno generirane primere, ki smo jih opisali v podpoglavju 4.2. Tukaj smo za Newtonovo metodo in Analitični TC pognali algoritem, kjer smo izračunali 5 pasov ob diagonali, da smo zmanjšali računsko zahtevnost. Kljub temu



Slika 9: Vpliv števila izračunanih pasov Hesseve matrike na konvergenco za 30 vmesnih točk. Vidimo lahko, da so 4 pasovi v tem primeru optimalno za najhitrejši izračun.



Slika 10: Vpliv števila izračunanih pasov Hesseve matrike na konvergenco za 40 vmesnih točk. Vidimo lahko, da je 5 pasov v tem primeru optimalno za najhitrejši izračun.

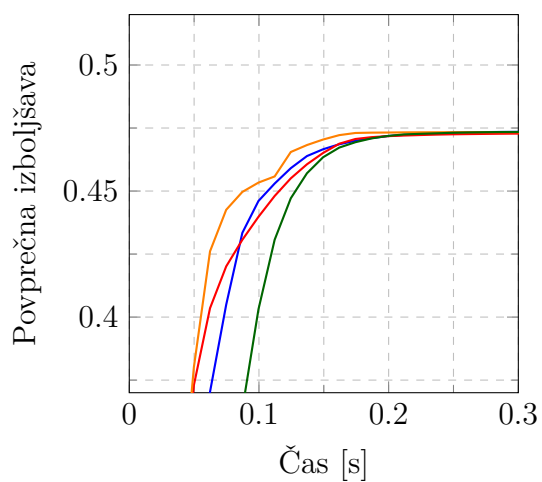
je na grafih razvidno, da v realnem času v obeh primerih Newtonova metoda po večini ni hitrejša od metode z gradientnim spustom. Na grafih, ki prikazujejo primere z 20 vmesnimi točkami ali več, ni prikazana numerična TC metoda, saj je v teh primerih njena konvergenca v realnem času tako počasna, da je pristala izven okvira grafov. Namreč, numerična TC metoda mora na vsakem koraku izvesti evalvacijo cenovne funkcije $(n + 1)$ -krat, kjer je n število segmentov trajektorije. V našem primeru to pomeni, da mora numerična TC metoda pri vsaki iteraciji rešiti $n + 1$ QP problemov, da lahko numerično izračuna odvode cenovne funkcije.

Pričakovano se čas izračuna povečuje z večjim številom segmentov trajektorije in v primeru s 40 segmenti presega 2 sekundi, kar je za optimizacijo trajektorij v realnem času preveč. Kljub temu pa ne opazimo bistveno povečane relativne razlike med gradientnim spustom in Newtonovo metodo. Prav tako je dobljeni rezultat na 40 točkah primerljiv z rezultati, ki jih navajajo v [sun2021fast], kjer je določen delež njihove kode napisan v programskem jeziku C++, kar omogoča hitrejši izračun. Tu priznavamo, da v [sun2021fast] dodatno uporabljajo neenakostne omejitve.

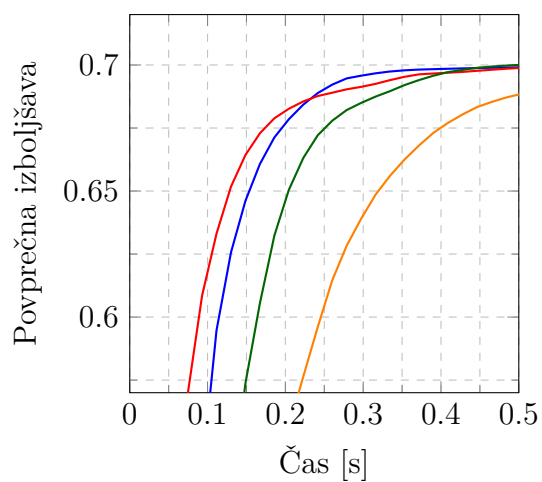
5 Zaključek

V tem magistrskem delu smo predstavili Newtonovo metodo za optimizacijo trajektorij za diferencialno ploščate sisteme. Pokazali smo, kako minimizacijo funkcionala z omejitvami pretvorimo v dvonivojski optimizacijski problem, kjer ima nižjenivojski problem strukturo QP z enakostnimi omejitvami. S pomočjo IIF smo pokazali, da lahko analitično izračunamo gradient in Hessovo matriko cenovne funkcije zgornjenivojskega problema. Z eksperimenti smo pokazali, da Newtonova metoda potrebuje manj iteracij za konvergenco kot metoda, ki uporablja samo gradient ali metoda, ki odvode izračuna numerično. Kljub temu v večini primerov Newtonova metoda v realnem času ni hitrejša od gradientnega spusta. Opisali smo tudi nekaj pristopov za pohitritev algoritma, ki izrabljajo strukturo problema.

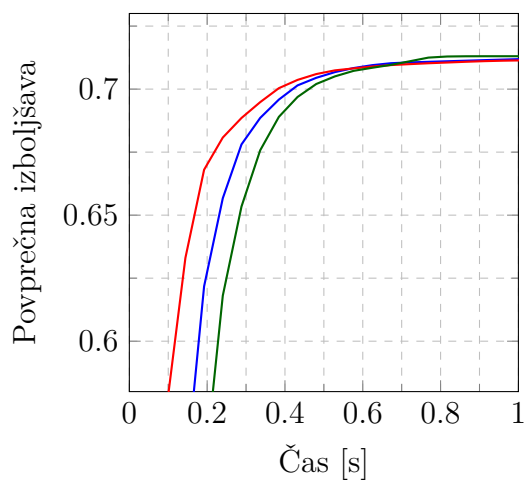
Ostaja mnogo poti za izboljšavo. Glavni izmed njih je vključitev neenakostnih omejitev v QP, kar bi omogočilo načrtovanje trajektorij z izogibanjem preprekam in omejitev maksimalnih hitrosti in pospeškov kvadrotorjev. To bi sicer lahko pripeljalo do težav pri veljavnosti IIF, vendar v [sun2021fast, amos2017optnet] že ponudijo rešitve. Dodatno bi bilo zanimivo raziskati vključitev nelinearnih omejitev, npr. omejitev stožca drugega reda, ki bi bile koristne za omejitve hitrosti in pospeška; za to bi bilo treba uporabiti algoritme za semidiefinitno programiranje, kot je npr. Clarabel [goulart2024clarabel] oz. CuClarabel [chen2024cuclarabel], ki deluje z grafičnimi procesnimi enotami. Nadalje, implementacija v nižjenivojskem jeziku, kot je C++, bi z veliko verjetnostjo bistveno izboljšala hitrost. Normalizacija časovnih intervalov, kot so jo predlagali v [mellinger2011minimum], bi pripomogla k večji stabilnosti metode. Uporaba metode ničelnega prostora za faktorizacijo matrike v enačbi (3.23) [rees2014null] bi lahko pripomogla k dodatni pohitritvi. Na strani implementacije bi bilo koristno dodati tudi nastavljive parametre za npr. začetne/končne odvode trajektorije. Še en zanimiv pristop k boljši hitrosti bi bil razvoj metode globokega učenja za aproksimacijo gradienta in Hessove matrike oz. optimizacijskega koraka; v preteklosti se je tak pristop izkazal za zelo učinkovit pri pospeševanju optimizacijskih algoritmov [andrychowicz2016learning].



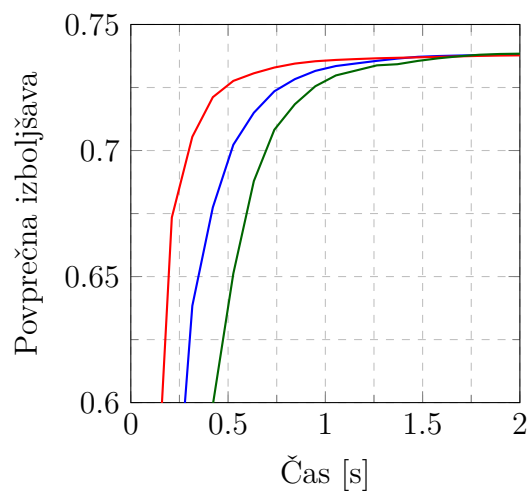
(a) 5 točk



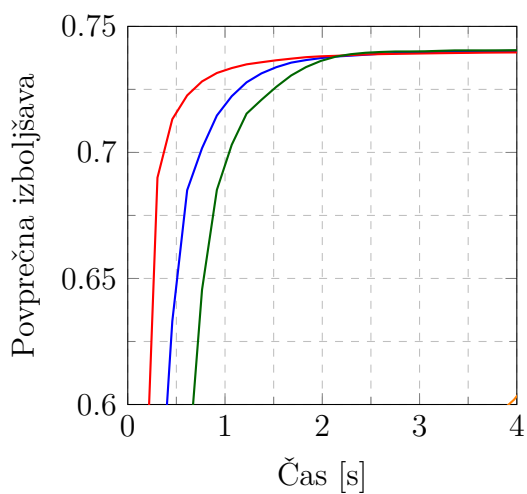
(b) 10 točk



(c) 20 točk

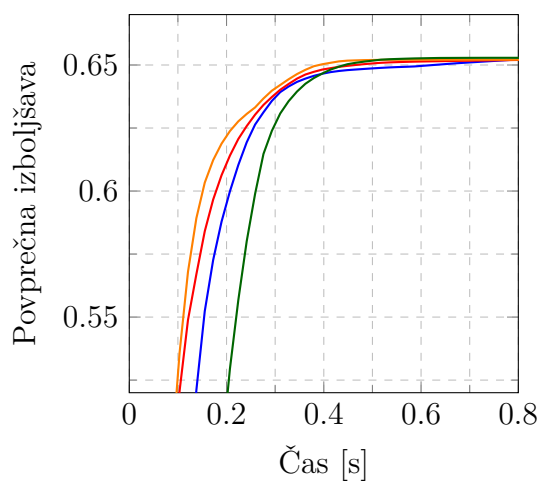


(d) 30 točk

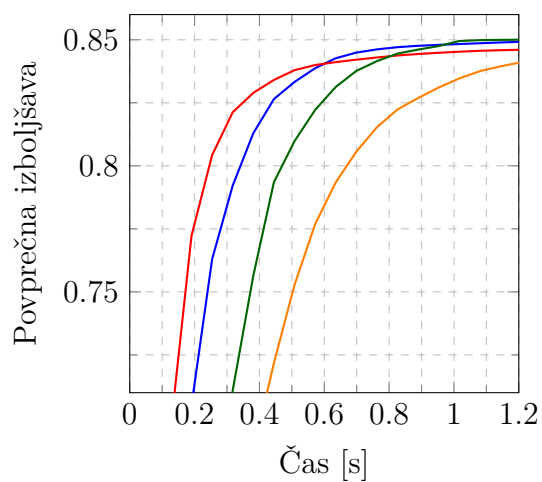


(e) 40 točk

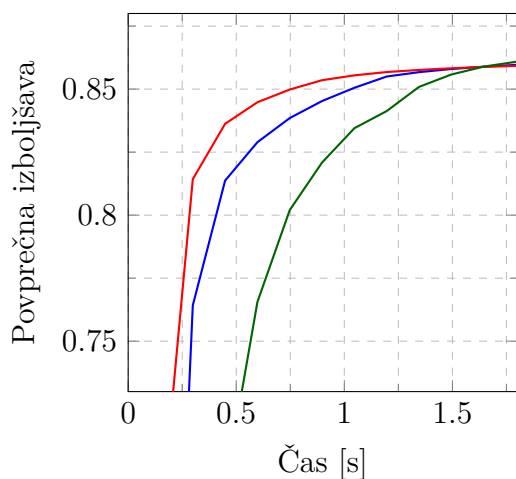
Slika 11: Časovna konvergenca z naključnimi začetnimi točkami



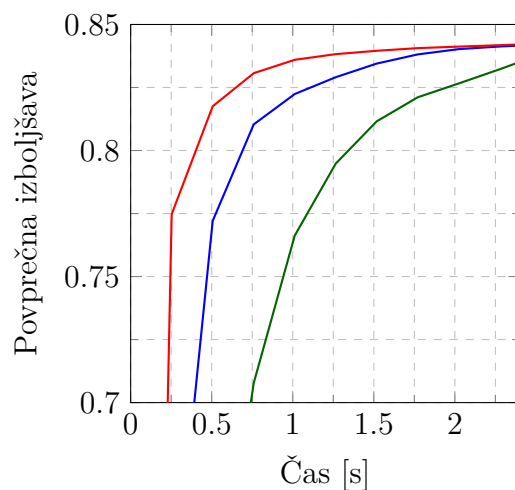
(a) 5 točk



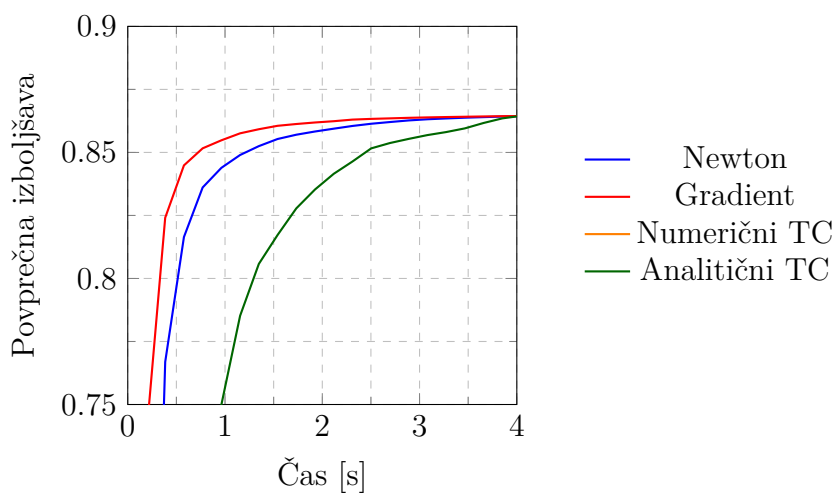
(b) 10 točk



(c) 20 točk



(d) 30 točk



(e) 40 točk

Slika 12: Časovna konvergenca z naključnimi točkami in premicami

A Primer

V tem dodatku predstavimo ilustrativen primer, ki služi boljšemu razumevanju opisane metode na konkretnem problemu. Najprej opišemo konstrukcijo QP s fiksnimi časi segmentov, nato pa še izračun komponent gradient cenovne funkcije $F(\mathbf{t})$. Na koncu si ogledamo še projekcijo gradienta na podprostor $\{\mathbf{t} : \mathbf{1}^\top \mathbf{t} = 0\}$. Izračun Hessove matrike izpustimo.

A.1 QP

Za boljše razumevanje konstrukcije QP s fiksnimi časi segmentov si oglejmo primer z $n = 2$ segmentoma, stopnjo polinomskih zlepkov $d = 4$, redom zveznosti $K = 2$ in optimizacijsko stopnjo odvoda $r = 3$. Optimizirali bomo trajektorijo v dveh dimenzijah, x in y , pri čemer bomo uporabili sledeče vmesne točke in čase segmentov:

$$\begin{aligned} P_0 &= (1, 0) \\ P_1 &= (1, 1) \\ P_2 &= (2, 0) \\ \mathbf{t} &= (1, 2) \end{aligned} \tag{A.1}$$

Torej, med točko P_0 in P_1 bomo potovali $1s$, med točko P_1 in P_2 pa $2s$. Sedaj bomo izračunali matriko $\mathbf{Q}$, ki bo imela bločno diagonalno strukturo:

$$\mathbf{Q} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 36 & 72 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 72 & 192 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 72 & 288 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 288 & 1536 & 0 \end{bmatrix} \tag{A.2}$$

Če eksplicitno zapišemo odvisnost od časov t_1 in t_2 , dobimo:

$$\mathbf{Q}(\mathbf{t}) = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 36t_1 & 72t_1^2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 72t_1^2 & 192t_1^3 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 36t_2 & 72t_2^2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 72t_2^2 & 192t_2^3 & 0 \end{bmatrix} \tag{A.3}$$

Matriko $\mathbf{A}$ izračunamo na podlagi strukture, opisane v prejšnjem podglavju. Za naš primer z dvema segmentoma in omejitvami do drugega odvoda dobimo:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 3 & 4 & 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 2 & 6 & 12 & 0 & 0 & -2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 2 & 4 & 8 & 16 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 4 & 12 & 32 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 12 & 48 \end{bmatrix} \quad (\text{A.4})$$

Če eksplicitno zapišemo odvisnost od časov t_1 in t_2 , dobimo:

$$\mathbf{A}(\mathbf{t}) = \begin{bmatrix} 1 \cdot 0^0 & 1 \cdot 0^1 & 1 \cdot 0^2 & 1 \cdot 0^3 & 1 \cdot 0^4 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 \cdot 0^0 & 2 \cdot 0^1 & 3 \cdot 0^2 & 4 \cdot 0^3 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 \cdot 0^0 & 6 \cdot 0^1 & 12 \cdot 0^2 & 0 & 0 & 0 & 0 & 0 \\ 1 \cdot t_1^0 & 1 \cdot t_1^1 & 1 \cdot t_1^2 & 1 \cdot t_1^3 & 1 \cdot t_1^4 & -1 \cdot 0^0 & -1 \cdot 0^1 & -1 \cdot 0^2 & -1 \cdot 0^3 & -1 \cdot 0^4 \\ 0 & 1 \cdot t_1^0 & 2 \cdot t_1^1 & 3 \cdot t_1^2 & 4 \cdot t_1^3 & 0 & -1 \cdot 0^0 & -2 \cdot 0^1 & -3 \cdot 0^2 & -4 \cdot 0^3 \\ 0 & 0 & 2 \cdot t_1^0 & 6 \cdot t_1^1 & 12 \cdot t_1^2 & 0 & 0 & -2 \cdot 0^0 & -6 \cdot 0^1 & -12 \cdot 0^2 \\ 0 & 0 & 0 & 0 & 0 & 1 \cdot 0^0 & 1 \cdot 0^1 & 1 \cdot 0^2 & 1 \cdot 0^3 & 1 \cdot 0^4 \\ 0 & 0 & 0 & 0 & 0 & 1 \cdot t_2^0 & 1 \cdot t_2^1 & 1 \cdot t_2^2 & 1 \cdot t_2^3 & 1 \cdot t_2^4 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \cdot t_2^0 & 2 \cdot t_2^1 & 3 \cdot t_2^2 & 4 \cdot t_2^3 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 \cdot t_2^0 & 6 \cdot t_2^1 & 12 \cdot t_2^2 \end{bmatrix} \quad (\text{A.5})$$

Vektorja $\mathbf{b}$ za x in y dimenziji sta:

$$\mathbf{b}_x = [1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 2 \ 0 \ 0]^\top, \quad \mathbf{b}_y = [0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0]^\top \quad (\text{A.6})$$

Pri tem smo z različnimi barvami poudarili bločno strukturo matrik in določili bloke $\mathbf{Q}_1(t_1)$, $\mathbf{Q}_2(t_2)$, $\mathbf{A}_1(0)$, $[\mathbf{A}_1(t_1) - \mathbf{A}_2(0)]$ in $\mathbf{A}_2(t_2)$.

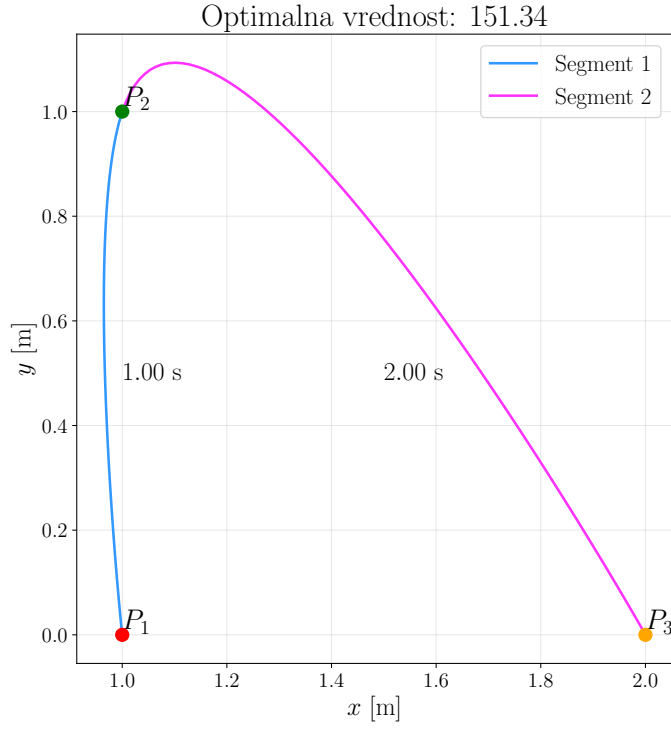
Sedaj lahko ločeno rešimo QP za x in y dimenzijo:

$$\begin{aligned} \min_{\mathbf{a}_x} \quad & \frac{1}{2} \mathbf{a}_x^\top \mathbf{Q} \mathbf{a}_x & \min_{\mathbf{a}_y} \quad & \frac{1}{2} \mathbf{a}_y^\top \mathbf{Q} \mathbf{a}_y \\ \text{p.p.} \quad & \mathbf{A} \mathbf{a}_x = \mathbf{b}_x & \text{p.p.} \quad & \mathbf{A} \mathbf{a}_y = \mathbf{b}_y \end{aligned} \quad (\text{A.7})$$

Rezultat optimizacije sta vektorja $\mathbf{a}_x$ in $\mathbf{a}_y$, ki vsebujeta koeficiente polinomskih zlepkov za x in y dimenzijo. S temi koeficienti lahko nato rekonstruiramo polinomske zlepke in jih združimo v celotno trajektorijo, ki je vidna na sliki 13.

A.2 Izračun gradienta

Za izračun gradienta cenovne funkcije $F(\mathbf{t})$ potrebujemo parcialne odvode matrike $\mathbf{Q}$ in $\mathbf{A}$ glede na časovne spremenljivke t_1 in t_2 . V tem razdelku predstavimo te parcialne odvode v simbolični in numerični obliki.



Slika 13: Trajektorija, dobljena z rešitvijo QP problema za $n = 2$ segmenta, $d = 4$, $K = 2$ in $r = 3$.

V simbolični obliki:

$$\partial_{t_1} \mathbf{Q}(\mathbf{t}) = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 36 & 144t_1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 144t_1 & 576t_1^2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (\text{A.8})$$

V numerični obliki pri $t_1 = 1, t_2 = 2$:

$$\partial_{t_1} \mathbf{Q}(1, 2) = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 36 & 144 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 144 & 576 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (\text{A.9})$$

V simbolični obliki:

$$\partial_{t_1} \mathbf{A}(\mathbf{t}) = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2t_1 & 3t_1^2 & 4t_1^3 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 6t_1 & 12t_1^2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 6 & 24t_1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (\text{A.10})$$

V numerični obliki pri $t_1 = 1, t_2 = 2$:

$$\partial_{t_1} \mathbf{A}(1, 2) = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 3 & 4 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 6 & 12 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 6 & 24 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (\text{A.11})$$

Sedaj lahko sestavimo parcialin odvod celotne KKT matrike $\mathbf{K}$:

$$\partial_{t_1} \mathbf{K}(\mathbf{t}) = \begin{bmatrix} \partial_{t_1} \mathbf{Q}(\mathbf{t}) & \partial_{t_1} \mathbf{A}(\mathbf{t})^\top \\ \partial_{t_1} \mathbf{A}(\mathbf{t}) & 0 \end{bmatrix}, \quad (\text{A.12})$$

s tem pa izračunamo tudi parcialne odvode implicitne funkcije γ_x (vektor polinomskih koeficientov in Lagrangeovih multiplikatorjev za dimenzijo x) glede na t_1 :

$$\partial_{t_1} \gamma_x(\mathbf{t}) = -\mathbf{K}(\mathbf{t})^{-1} (\partial_{t_1} \mathbf{K}(\mathbf{t}) \gamma_x(\mathbf{t})). \quad (\text{A.13})$$

Spomnimo, da v implementaciji inverza matrike $\mathbf{K}$ nikoli ne izračunamo eksplisito, temveč rešujemo sistem enačb.

Za naš primer pri $t_1 = 1, t_2 = 2$ lahko izračunamo numerično vrednost parcialnega odvoda $\partial_{t_1} \gamma_x(1, 2)$. Ker je $\gamma_x = (\bar{\mathbf{a}}, \bar{\mu})$, lahko rezultat razdelimo na komponenti:

$$\partial_{t_1} \bar{\mathbf{a}}_x(1, 2) = \begin{bmatrix} 0.00000 \\ 0.00000 \\ 0.00000 \\ 0.44358 \\ -0.77698 \\ 0.00000 \\ 0.22201 \\ -0.33302 \\ 0.16651 \\ -0.02775 \end{bmatrix}, \quad \partial_{t_1} \bar{\mu}_x(1, 2) = \begin{bmatrix} 123.90191 \\ 56.61241 \\ 4.99130 \\ -123.90191 \\ 19.31748 \\ 5.66212 \\ -143.88543 \\ 19.98352 \\ -20.64956 \\ 6.99419 \end{bmatrix} \quad (\text{A.14})$$

S pomočjo enačbe (3.29) lahko sedaj izračunamo komponento gradienta glede na t_1 :

$$\partial_{t_1} F_x(1, 2) = \bar{\mathbf{a}}_x(1, 2)^\top \mathbf{Q}(\partial_{t_1} \bar{\mathbf{a}}_x(1, 2)) + \frac{1}{2} \bar{\mathbf{a}}_x(1, 2)^\top (\partial_{t_1} \mathbf{Q}(1, 2)) \bar{\mathbf{a}}_x(1, 2) = -9,98352669 \quad (\text{A.15})$$

Na analogen način lahko izračunamo tudi $\partial_{t_2} F_x$, $\partial_{t_1} F_y$ in $\partial_{t_2} F_y$, s čimer dobimo celoten gradient $\nabla F(\mathbf{t})$:

$$\nabla F(\mathbf{t}) = (\partial_{t_1} F_x, \partial_{t_2} F_x) + (\partial_{t_1} F_y, \partial_{t_2} F_y). \quad (\text{A.16})$$

Ko vse te korake izračunamo, dobimo vrednost gradienta v točki $\mathbf{t} = (1, 2)$, ki je:

$$\nabla F(1, 2) = (-495, 5232, -130, 6149) \quad (\text{A.17})$$

Opazimo lahko, da sta obe komponenti gradienta negativni, kar pomeni, da bi se s povečanjem časov t_1 in t_2 lahko znižala vrednost cenovne funkcije $F(\mathbf{t})$. To je smiselno, saj daljši časovni intervali omogočajo manj agresivne trajektorije, kar vodi do nižje vrednosti funkcije $F(\mathbf{t})$. Vendar pa bi, če bi uporabili ta gradient v optimizacijskem algoritmu, prekršili omejitev $\mathbf{1}^\top \mathbf{t} = T$ oz. $t_1 + t_2 = T$, zato moramo uporabiti projekcijo gradienta na hiperravnino. Ta postopek bomo na tem primeru predstavili v naslednjem razdelku.

A.3 Projekcija gradienta in gradientni korak

Določimo $T = 3$, kar pomeni, da imamo omejitev $t_1 + t_2 = 3$. Zato želimo gradient $\nabla F(\mathbf{t})$ projicirati na hiperravnino, določeno z normalnim vektorjem $\mathbf{1} = (1, 1)$. Projekcija gradienta $\nabla F(\mathbf{t})$ na to hiperravnino se izračuna po formuli (??), njen rezultat pa je:

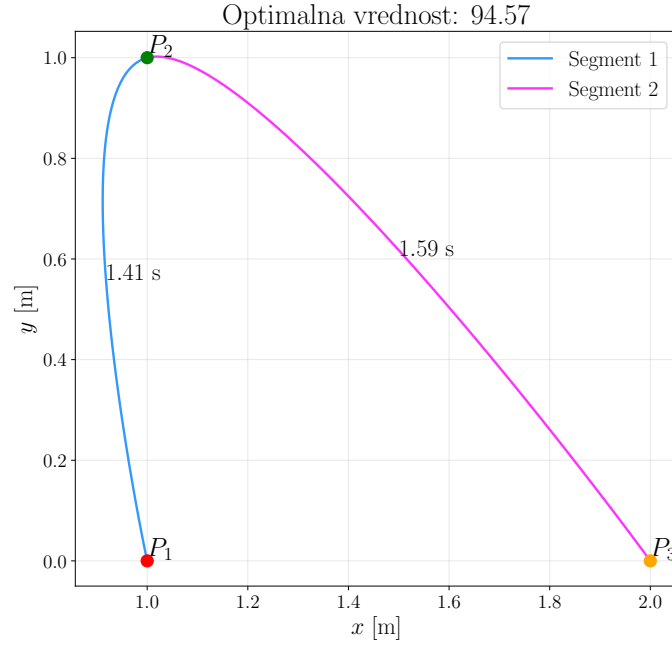
$$\nabla_{\text{proj}} F(1, 2) = (-182.45414851, 182.45414851) \quad (\text{A.18})$$

V implementaciji ta gradient še normaliziramo, da se izognemo prevelikim korakom, in izvedemo gradientni korak z velikostjo $\alpha = 1$:

$$\mathbf{t}_1 = \mathbf{t}_0 - \alpha \frac{\nabla_{\text{proj}} F(\mathbf{t}_0)}{\|\nabla_{\text{proj}} F(\mathbf{t}_0)\|} = (1, 2) - (-0.70710678, 0.70710678) = (1.70710678, 1.29289322) \quad (\text{A.19})$$

B Podrobnosti implementacije

V tem dodatku se posvetimo podrobnostim implementacije hitrega izračuna analitičnega gradienta in Hessove matrike cenovne funkcije $F(\mathbf{t})$. V prejšnjih poglavjih smo zavoljo jasnosti odvode različnih funkcij in matrik opisovali po komponentah (na primer, $\partial_{t_i} F$, $\partial_{t_i t_j} \bar{\mathbf{a}}$, $\partial_{t_i t_i} \mathbf{Q}$, ...), vendar pa je v implementaciji smiselno uporabiti vektorizacijo in vektorske operacije, da se izognemo zankam in nepotrebnemu računanju z ničelnimi bloki. V tem dodatku bomo opisali, kako smo z vektorizacijo izračunali Hessovo matriko $\nabla^2 F(\mathbf{t})$ in gradient $\nabla F(\mathbf{t})$. Najprej predstavimo, kako smo prestrukturirali vhodne komponente algoritma 1 v tenzorje brez ničelnih



Slika 14: Projekcija gradienta na hiperravnino $t_1 + t_2 = T$.

blokov, nato pa s pomočjo Einsteinove sumacijske notacije opišemo, kako smo izračunali gradient in Hessovo matriko. Najprej pa na kratko predstavimo Einsteinovo sumacijsko notacijo.

B.1 Einsteinova sumacijska notacija

Einsteinova sumacijska notacija (Einstein summation notation) je konvencija v tenzorskem računu, ki omogoča bolj kompakten zapis sumacij. Osnovno pravilo je, da če se isti indeks pojavi dvakrat v produktu – enkrat kot spodnji in enkrat kot zgornji indeks – ta indeks implicitno indicira sumacijo čez vse vrednosti tega indeksa.

Formalno: če imamo izraz oblike $a_i b^i$, to pomeni $\sum_i a_i b^i$.

Primeri:

- Skalarni produkt: $\mathbf{u} \cdot \mathbf{v} = u^i v_i = \sum_{i=1}^n u^i v_i$
- Matrično-vektorski produkt: $(\mathbf{A}\mathbf{x})_i = A_{ij} x^j = \sum_{j=1}^n A_{ij} x^j$
- Matrični produkt: $(\mathbf{A}\mathbf{B})_{ij} = A_{ik} B_{kj} = \sum_{k=1}^n A_{ik} B_{kj}$
- Sled matrike: $\text{tr}(\mathbf{A}) = A_{ii} = \sum_{i=1}^n A_{ii}$

V tem dodatku uporabljamo Einsteinovo notacijo za opis vektoriziranih operacij pri izračunu gradienta in Hessove matrike. To nam omogoča, da kompaktno zapišemo operacije na tenzorjih, ki v implementaciji ustrezajo vektorskim operacijam brez eksplicitnih zank.

B.2 Vektorizacija vhodnih podatkov

Kot vhodne podatke algoritma 1 imamo matriko $\mathbf{Q}(\mathbf{t})$, matriko $\mathbf{A}(\mathbf{t})$, vektorja $\mathbf{a}^*$ in $\boldsymbol{\mu}^*$, nato pa izračunamo tudi parcialne odvode $\partial_{t_i} \mathbf{Q}$, $\partial_{t_i} \mathbf{A}$, $\partial_{t_i t_j} \mathbf{Q}$ in $\partial_{t_i t_j} \mathbf{A}$. Vse

te matrike imajo redko strukturo, saj so sestavljene iz blokov, kjer je večina blokov ničelnih. V implementaciji smo te matrike prestrukturirali v tenzorje, kjer so shranjeni le neničelni bloki, kar nam omogoča, da z vektorskimi operacijami izračunamo vse parcialne odvode in komponente gradienta in Hessove matrike v enem klicu funkcije, brez zank po komponentah. Ti tenzorji so:

- $\mathcal{Q}$: tenzor velikosti $(n, d+1, d+1)$, kjer n je število segmentov trajektorije in d je stopnja polinoma. Ta tenzor vsebuje neničelne bloke matrike $\mathbf{Q}$. Velja $\mathcal{Q}_i = \mathbf{Q}_i$ za $i = 1, \dots, n$.
- $\mathcal{Q}'$: tenzor velikosti $(n, d+1, d+1)$, kjer n je število segmentov trajektorije in d je stopnja polinoma. Ta tenzor vsebuje neničelne bloke parcialnih odvodov matrike $\mathbf{Q}$. Velja $\mathcal{Q}'_i = \partial_{t_i} \mathbf{Q}_i$ za $i = 1, \dots, n$.
- $\mathcal{A}'$: tenzor velikosti $(n, q+2, d+1)$, kjer n je število segmentov trajektorije, q je stopnja minimiziranega odvoda in d je stopnja polinoma. Ta tenzor vsebuje neničelne bloke parcialnih odvodov matrike $\mathbf{A}$. Velja $\mathcal{A}'_i = \partial_{t_i} \mathbf{A}_i$ za $i = 1, \dots, n$.¹
- $\mathcal{Q}''$: tenzor velikosti $(n, d+1, d+1)$, kjer n je število segmentov trajektorije in d je stopnja polinoma. Ta tenzor vsebuje neničelne bloke parcialnih odvodov drugega reda matrike $\mathbf{Q}$. Velja $\mathcal{Q}''_i = \partial_{t_i t_i} \mathbf{Q}_i$ za $i = 1, \dots, n$.
- $\mathcal{A}''$: tenzor velikosti $(n, q+2, d+1)$, kjer n je število segmentov trajektorije, q je stopnja minimiziranega odvoda in d je stopnja polinoma. Ta tenzor vsebuje neničelne bloke parcialnih odvodov drugega reda matrike $\mathbf{A}$. Velja $\mathcal{A}''_i = \partial_{t_i t_i} \mathbf{A}_i$ za $i = 1, \dots, n$.¹
- $\mathbf{a}$: tenzor velikosti $(n, d+1)$, kjer n je število segmentov trajektorije in d je stopnja polinoma. Ta tenzor vsebuje neničelne bloke vektorja $\mathbf{a}^*$, kjer velja $\mathbf{a}_i = \mathbf{a}_i^*$ za $i = 1, \dots, n$.
- $\boldsymbol{\mu}$: tenzor velikosti $(n, q+2)$, kjer n je število segmentov trajektorije in q je stopnja minimiziranega odvoda. Ta tenzor vsebuje neničelne bloke vektorja $\boldsymbol{\mu}^*$, kjer velja $\boldsymbol{\mu}_i = \boldsymbol{\mu}_i^*$ za $i = 1, \dots, n$.
- $\mathbf{a}'$: tenzor velikosti $(n, n, d+1)$, kjer n je število segmentov trajektorije in d je stopnja polinoma. Ta tenzor vsebuje parcialne odvode vektorja $\mathbf{a}^*$, kjer velja $\mathbf{a}'_{ij} = \partial_{t_i} \mathbf{a}_j^*$ za $i, j = 1, \dots, n$.
- $\boldsymbol{\mu}'$: tenzor velikosti $(n, n, q+2)$, kjer n je število segmentov trajektorije in q je stopnja minimiziranega odvoda. Ta tenzor vsebuje parcialne odvode vektorja $\boldsymbol{\mu}^*$, kjer velja $\boldsymbol{\mu}'_{ij} = \partial_{t_i} \boldsymbol{\mu}_j^*$ za $i, j = 1, \dots, n$.

¹Striktno gledano imata bloka $\partial_{t_n} A_n$ in $\partial_{t_n t_n} A_n$ eno vrstico manj od ostalih blokov ($q+1$ vrstic, saj imamo pri ostalih blokih dodatno omejitve, ki določa skladanje končne koordinate z začetno v naslednjem segmentu), vendar pa smo v implementaciji dodali dodatno ničelno vrstico, da so vsi bloki enake velikosti in da lahko uporabimo vektorske operacije brez skrbi o različnih velikostih blokov.

B.3 Izračun gradienta

Sedaj lahko z Einsteinovo sumacijsko notacijo zapišemo sledečo matriko:

$$\mathbf{B}_{grad} = - \begin{bmatrix} \text{blkdiag}(\mathcal{Q}'_{ij}a_j + \mathcal{A}'_{kij}\boldsymbol{\mu}_k) \\ \mathbf{0} \\ \text{blkdiag}(\mathcal{A}'_{ikj}a_j) \end{bmatrix} \quad (\text{B.1})$$

kjer i -ti stolpec matrike $\mathbf{B}_{grad}$ predstavlja desno stran linearne sistema, da izračunamo $\partial_{t_i}\gamma$. $\mathbf{0}$ je ničelna matrika velikosti $q + 1 \times n$. Simbol blkdiag označuje operacijo bločne diagonalizacije iz tridimenzionalnega tenzorja dimenzij $k \times l \times m$ v bločno diagonalno matriko velikosti $kl \times km$, kjer je i -ti element tenzorja (po prvi dimenziji) i -ti diagonalni blok v matriki. Z izračunom matrike $\mathbf{B}_{grad}$ se tako izognemo zankam po komponentah, kot smo to opisali v podpoglavju 3.6.2, in povečamo učinkovitost izračuna gradienta. Ko imamo matriko $\mathbf{B}_{grad}$, lahko rešimo sledeči sistem enačb:

$$\mathbf{K}\partial\gamma = \mathbf{B}_{grad}, \quad (\text{B.2})$$

kjer je

$$\partial\gamma = \begin{bmatrix} \partial_{t_1}\gamma & \partial_{t_2}\gamma & \cdots & \partial_{t_n}\gamma \end{bmatrix}. \quad (\text{B.3})$$

Matriko $\partial\gamma$ lahko nato uporabimo za izračun $\nabla F(\mathbf{t})$, pa tudi kasneje za izračun desne strani linearne sistema za izračun Hessove matrike $\nabla^2 F(\mathbf{t})$. Tudi Hessovo matriko $\nabla^2 F(\mathbf{t})$ izračunamo s pomočjo tenzorjev in Einsteinove sumacijske notacije, vendar pa je ta izračun nekoliko bolj zapleten, zato bralec prepuščamo, da si ga ogleda v implementaciji (specifično v funkciji `build_hess_rhs` v datoteki `solver.py`).